



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE GRADO

Renderización 2D con OpenGL

Realizado por
Iván Barbero Lay

Para la obtención del título de
Grado en Ingeniería Informática - Tecnologías Informáticas

Dirigido por
María José Morón Fernández

En el departamento de
Arquitectura y Tecnología de Computadores

Convocatoria de Octubre, curso 2025/26

Aquí la dedicatoria del trabajo

Agradecimientos

Quiero agradecer a mi tutora de *TFG* por su confianza y ánimos, a la vez que su guía y consejos.

También, quiero agradecer a mis compañeros de facultad y mi pareja, que me dieron siempre los mejores momentos y suavizaron los peores.

Resumen

Este trabajo explora el mundo de los gráficos por ordenador, sus tecnologías, técnicas y métodos. En el proyecto se desarrollarán varias bibliotecas que faciliten la programación con *APIs* gráficas, concretamente, *OpenGL*, además de proveer sistemas de animación y creación de polígonos. Finalmente, se analizará el rendimiento de estos sistemas recogiendo datos y convirtiéndolos en gráficas.

Palabras clave: Computación de gráficos, optimización a bajo nivel, bibliotecas.

Abstract

This project explores the world of computer graphics, its technologies, techniques and methods. Various libraries will be developed that ease graphical *API* programming, specially with *OpenGL*, provide animation systems and polygon creation. Finally, efficiency and performance of these programs will be measured and converted to graphs.

Keywords: Computer Graphics, Low-level optimization, libraries.

Índice general

1 Introducción 1

2 Metodología y Planificación 3
2.1. Metodología 3
2.2. Fases y Planificación 4
2.2.1. Fase 1: Estudio sobre los sistemas de renderizado y análisis del proyecto 4
2.2.2. Fase 2: Configuración del entorno de trabajo, bibliotecas propias y sistema de animación 4
2.2.3. Fase 3: Pruebas y optimización 6
2.3. Diagrama de Gantt 6

3 Marco Conceptual y Estado de la técnica 8
3.1. Antialiasing 8
3.2. APIs de Renderización 10
3.3. Implementaciones existentes 10
3.3.1. Godot 10
3.3.2. Blender 11
3.3.3. GTK y GSK 11
3.3.4. Unity 12
3.3.5. Comparación 12

4 Análisis del problema 14
4.1. Requisitos 14
4.1.1. Requisitos de información 14
4.1.2. Requisitos funcionales 15
4.1.3. Requisitos no funcionales 15
4.1.4. Requisitos hardware 15

5 Lenguaje y Tecnologías Empleadas 16
5.1. Lenguaje C 16
5.2. Tecnologías de renderizado 17
5.2.1. OpenGL 17
5.2.2. GLSL 26
5.3. Herramientas de soporte 28
5.3.1. CMake 28
5.3.2. GLEW 28
5.3.3. Freeglut 29
5.3.4. Git 30

6 Diseño del sistema 31
6.1. Leyenda 31
6.2. Shaders 33
6.2.1. Decisiones de diseño 33
6.2.2. API pública 34
6.3. Errores 34
6.3.1. Decisiones de diseño 35
6.3.2. API pública 37
6.4. Misceláneo 37
6.4.1. Decisiones de diseño 38
6.4.2. API pública 38
6.5. Polígonos 38
6.5.1. Decisiones de diseño 38
6.5.2. API pública 39
6.6. Animaciones 40
6.6.1. Decisiones de diseño 43
6.6.2. API pública 46

7 Implementación 47
7.1. Herramientas 47
7.2. Desarrollo de bibliotecas 47
7.2.1. Shaders 48
7.2.2. Errores 48
7.2.3. Misceláneo 49
7.2.4. Polígonos 49
7.2.5. Animaciones 49
7.2.6. Ejemplo de uso 52
7.2.7. Demostración 56
7.3. Conclusiones 58

8 Pruebas 59
8.1. Pruebas de rendimiento de creación de polígonos 59
8.1.1. Problemas encontrados 60
8.1.2. Resultados 60
8.2. Pruebas de rendimiento de animaciones 65
8.2.1. Problemas encontrados 66
8.2.2. Resultados 66
8.3. Análisis de memoria 71
8.4. Análisis de CPU 74
8.5. Conclusiones 75

9 Conclusiones 76
9.1. Líneas futuras 76

Bibliografía 77

Índice de figuras

2.1. Diagrama de Gantt detallado del proyecto 7

3.1. Comparación de dos modelos, uno sin antialising y otro con antialiasing. 9
3.2. Colores de un triángulo siendo evaluados sin antialiasing. 9
3.3. Colores de un triángulo siendo evaluados con antialiasing MSAA . . 10

5.1. Diagrama de Khronos que describe el flujo de shaders en OpenGL. . 22

6.1. Diagrama general de las bibliotecas y sus relaciones. 32
6.2. Leyenda de los diagramas de las bibliotecas. 32
6.3. Diagrama de la biblioteca de shaders. 33
6.4. Diagrama de la biblioteca de errores. 35
6.5. Diagrama de la biblioteca de funciones misceláneas de C. 37
6.6. Ecuación de las coordenadas polares en dos dimensiones 39
6.7. Ecuación del ángulo interno en un polígono 39
6.8. Diagrama de la biblioteca de polígonos. 40
6.10. Diagrama de la biblioteca de animaciones 42

7.1. Diagrama de la función por defecto por fotograma. 52
7.2. Diagrama del programa de ejemplo. 53
7.3. Diagrama de inicialización de shaders en el programa de ejemplo. . . 54
7.4. Diagrama de inicialización de datos para OpenGL. 55
7.5. Diagrama de inicialización de los datos para la animación. 56

8.1. Captura de pantalla de la prueba del polígono de 3 a 1000 vértices . . 61
8.2. Gráfica que representa el rendimiento de la *CPU* en la prueba del polígono durante su creación. 62
8.3. Gráficas que representan el rendimiento de la *GPU* y la *CPU* en la prueba del polígono durante la renderización. 64
8.4. Captura de pantalla de la prueba de la animación de 1024 polígonos en una matriz cuadrada. 67
8.5. Gráfica que representa el rendimiento de la *CPU* en la prueba de animación durante la creación de los polígonos. 68
8.6. Gráficas que representan el rendimiento de la *GPU* y la *CPU* en la prueba de animación durante la renderización. 69
8.7. Gráfica que representa el uso de memoria en la prueba del polígono . 72
8.8. Gráfica que representa el uso de memoria en la prueba de animación 73

Índice de tablas

3.1. Comparación de motores de renderización 13

4.1. Tabla de requisitos 14

8.1. Resultados de la creación de vértices en la prueba del polígono . . . 63
8.2. Resultados de la renderización en la prueba del polígono: **GPU** . . . 64
8.3. Resultados de la renderización en la prueba del polígono: **CPU** . . . 65
8.4. Resultados de la creación de polígonos en la prueba de animación (**CPU**) 68
8.5. Resultados de la renderización en la prueba del polígono: **GPU** . . . 70
8.6. Resultados de la renderización en la prueba del polígono: **CPU** . . . 70
8.7. Comparación de los resultados de la herramienta *perf* durante la ejecución de las pruebas. 74

Índice de extractos de código

5.1. Ejemplo de puntero a función en C 16
5.2. Ejemplo de objeto en OpenGL 18
5.3. Ejemplo de creación de los objetos VBO y VAO. 19
5.4. Ejemplo de uso de los objetos VBO y VAO. 20
5.5. Ejemplo de creación de EBO. 20
5.6. Ejemplo de uso de EBO. 21
5.7. Ejemplo de uso de variables Uniform. 23
5.8. Ejemplo de configuración de atributos. 24
5.9. Ejemplo de compilación de un shader 25
5.10. Ejemplo de creación de un programa 26
5.11. Ejemplo de Vertex Shader 27
5.12. Ejemplo de Fragment Shader 28
5.13. Ejemplo de inicialización de GLEW 29
5.14. Ejemplo de inicialización de Freeglut 30
6.1. Prototipos de la biblioteca de shaders 33
6.2. Prototipos de la biblioteca de errores 36
6.3. Prototipos de la biblioteca miscelánea 38
6.4. Prototipos de la biblioteca de polígonos 39
6.5. Prototipos de la bibliteca de animación; parte 1 45
6.6. Prototipos de la biblioteca de animación; parte 2 46

1. Introducción

El término conocido como gráficos por ordenador, o *Computer Graphics*, fue acuñado en 1960 por un empleado de Boeing, llamado William Fetter, que a su vez lo atribuyó a su compañero Verne Hudson. Es una tecnología relativamente reciente que crece y cambia rápidamente. El término puede usarse para describir la representación de imágenes en un ordenador y rodea el aspecto visual en un entorno digital.

En este trabajo se explorarán los denominados *renderer*, en concreto la renderización en dos dimensiones. Existen muchas definiciones, aunque aquí citaré una que creo bastante acertada. El glosario de [1] lo definió así:

> 2D rendering creates visuals from flat graphic elements through specialized techniques like sprite batching, tile-based systems, and optimized draw calls, often enhanced with modern lighting and shader effects despite the two-dimensional nature.

Este tema puede ser abrumador para el estudiante que lo toma sin conocimiento previo, por lo que este trabajo puede servir de iniciación para muchos de los conceptos que asolan este escabroso terreno. En su debida medida, se trazará una ruta de aprendizaje, aunque asumiendo algunos conocimientos básicos, sobre todo en el lenguaje de programación *C* [2]; será el lenguaje usado durante todo el documento, usando *C++* en contadas ocasiones.

Existen muchos tipos de *toolkits* que facilitan el renderizado de imágenes sin entrar en peliagudos sistemas gráficos usando directamente las *APIs* de gráficos como *OpenGL* [3], cuya *API* se usará en este trabajo, o *Vulkan* [4]. Por ejemplo, si uno quisiera desarrollar una aplicación de escritorio, podría utilizar *GTK* [5], que ofrece una biblioteca extensa. Si se pretende diseñar un modelo o un escenario puede usarse *Blender* [6], que permite crear el modelo y renderizarlo sin escribir una sola línea de código. O incluso, si se desea crear un videojuego, *Godot* [7] o *Unity* [8]. Todos estos entornos hacen uso de *APIs* de gráficos y de sistemas de renderización, aunque lo ocultan al usuario para facilitarle el desarrollo.

Cabe destacar el énfasis en el software libre respecto a las tecnologías usadas y que el entorno de desarrollo tendrá su foco en el sistema operativo *GNU/Linux* [9].

Objetivos Los objetivos generales del proyecto, definidos a alto nivel, son los siguientes:

- Comprensión del funcionamiento de *OpenGL* y de un renderizador.
- Establecer un entorno de trabajo para el desarrollo de los programas, con capacidad de modularización y gestión de dependencias.
- Desarrollar una biblioteca que facilite el uso de *OpenGL*: procesamiento de *shaders*, manejo de animaciones, gestión de errores y creación de polígonos.

- Aplicar el renderizado a una aplicación que genere formas geométricas y le aplique transformaciones para simular movimiento.
- Analizar los métodos de renderizado: la integridad de los resultados, su eficacia y su eficiencia.
- Optimización de los algoritmos y conseguir escalabilidad.

Estructura de la memoria Las fases que atravesará el proyecto pueden resumirse así:

1. Primero, se describirá la metodología seguida y la estructura utilizada durante el desarrollo del proyecto 2.
2. Se hará una introducción a la renderización y sus fundamentos, sobre sus conceptos y ejemplos de funcionamiento, además de *APIs* e implementaciones existentes 3.
3. Se hará un análisis de requisitos con 4 tipos: **información**, **funcional**, **no funcional** y **hardware**, donde se describen las necesidades que el proyecto debe satisfacer 4.
4. Se describirán los lenguajes de programación y tecnologías utilizados con el fin de desarrollar el proyecto, haciendo especial hincapié en *OpenGl* y en su funcionamiento 5.
5. Se presentará el diseño del sistema: las bibliotecas propias aquí implementadas, sus usos y complicaciones. Se representarán sus diseños en diagramas y se comentarán a alto nivel 6.
6. Se describirá la implementación del diseñado previamente presentado, entrando en detalle sobre cada función y estructura de datos y cómo deberían ser utilizados desde la perspectiva del usuario 7.
7. Se realizarán pruebas sobre el *software* desarrollado, analizando los resultados y representándolos mediante gráficas 8.
8. Finalmente, se realizarán unas conclusiones sobre el trabajo expuesto y se comentarán líneas futuras del proyecto: ideas sin finalizar o ideas a largo plazo.

2. Metodología y Planificación

En este capítulo se describen las etapas previas al análisis, fundamentales para garantizar el correcto desarrollo del proyecto.

Se comienza en el apartado 2.1 describiendo la metodología usada. Seguidamente, en 2.2, se presentan las fases en las que se ha abordado el desarrollo del trabajo y la planificación temporal seguida.

2.1. Metodología

Git El desarrollo del proyecto ha sido gestionado con *Git* [10], alojado en un repositorio[11] de *Gitlab* [12]. La metodología ha consistido en implementar funcionalidades de cada área en una rama distinta, para luego ser unida a la rama principal. De esta manera, es más sencillo detectar dónde los errores fueron generados y localizarlos.

Sistema operativo Todo el desarrollo tuvo lugar en el sistema operativo *GNU/Linux* [9], concretamente la distribución *Debian* [13]. A modo de diario personal, el progreso fue siendo guardado semanalmente en un documento *Asciidoc* [14], que ha servido de bitácora y muy útil para la memoria.

Dependencias y compilación El flujo de compilación y la gestión de dependencias es el objetivo de utilizar *CMake*. Con esta tecnología, es sencillo modularizar el proyecto, añadiendo dependencias según las necesidades del programa y creando una estructura de proyecto ordenada. Incluso, es simple compilar bibliotecas propias y enlazarlas posteriormente al código. Las ventajas frente a compilar manualmente son abrumadoras.

Fuentes La mayoría de la documentación que se ha necesitado proviene directamente de los autores de *OpenGL*, *Khronos* [15]. Tanto el libro como su página web donde detallan minuciosamente las funciones fueron de gran utilidad. El resto de documentación proviene de sus respectivos autores, buscando información por la red cuando fuera necesario.

Bibliotecas Respecto a la implementación, se ha intentado extraer el máximo número de funciones posible a bibliotecas propias según crece el código. De esta manera, se divide la carga y es más fácil discernir errores y aumenta la reutilización de código.

2.2. Fases y Planificación

A continuación, se describen las fases en las que se ha abordado el proyecto. El proceso de elaboración de documentación va implícito en cada fase, evolucionando a lo largo del proyecto. En la fase 1 2.2.1 se analizan los sistemas de renderizado, especialmente *OpenGl*, y se realiza un análisis sobre la estrategia que se llevará a cabo para desarrollar el programa y las bibliotecas. En la fase 2 2.2.2 se configura el entorno de trabajo y se implementan las bibliotecas y el programa principal. En la fase 3 2.2.3 se realizan pruebas a partir del *software* desarrollado y gráficas con los resultados obtenidos.

2.2.1. Fase 1: Estudio sobre los sistemas de renderizado y análisis del proyecto

Estudio sobre OpenGL y conceptos de renderización Primero, es necesario comprender cómo funciona *OpenGL* y los sistemas de renderización desde los conceptos básicos, es el primer paso esencial. Para entender esto, están a disposición manuales y guías sobre gráficos por ordenador por doquier, aunque para este trabajo los de *Khronos* son particularmente interesantes.

Investigación sobre herramientas de soporte También, investigar algunas bibliotecas que facilitan el desarrollo, como pueden ser *GLUT* y *GLEW*.

Análisis del programa principal y bibliotecas propias esenciales En esta etapa, es necesario analizar el programa principal y posibles dependencias o bibliotecas.

2.2.2. Fase 2: Configuración del entorno de trabajo, bibliotecas propias y sistema de animación

Configuración de entorno Para conseguir establecer un entorno de trabajo organizado con *CMake* [16], se divide el proyecto en varias carpetas:

- ***lib***, para lo relacionado con las bibliotecas propias y externas.
- ***src***, para lo relacionado con el código fuente principal de los programas.
- ***shaders***, para archivos de tipo *shader* de *OpenGL*.
- ***tests***, para la gestión de los resultados de las pruebas: datos y *script* de *R* y *gnuplot* para analizarlos.

Cada una de las carpetas contiene código *CMake*, profundizando en las carpetas según el nivel de complejidad que se requiera. Por ejemplo, dentro de *src* existen varios directorios con las áreas implementadas, como *movement* o *polygon*, y dentro de estos se declara la configuración de compilación de los programas relacionados con el movimiento.

Bibliotecas propias Con el objetivo de facilitar tareas de *OpenGL*, se ha optado por crear una biblioteca cuyo código se encontraría dentro del directorio *lib*. Este paso debería llevarse a cabo simultáneamente con el desarrollo del código principal, pues sería conveniente extraer las funciones y colocarlas en la biblioteca según aumente el programa.

- Una biblioteca de *shaders*, que aisla en una caja negra todo el proceso de un *shader*. Debería ser capaz de, recibiendo los contenidos del *shader*, devolver un *programa*, así llamado en la jerga de *OpenGL*. El usuario, entonces, no necesitaría conocer los mecanismos de este proceso.
- Una biblioteca de polígonos, que contiene funciones que creen formas geométricas, útiles para las pruebas de los programas desarrollados. En concreto, se centrará en crear polígonos regulares de cualquier número de lados. Son fáciles de crear y son un instrumento conveniente para estimar rendimiento según número de vértices.
- Una biblioteca de animaciones, que es el principal objetivo, ya que reúne todo lo anterior creado y permite crear una secuencia de movimientos para los vértices en pantalla. Esto pone a prueba el rendimiento de los sistemas, ya que es necesaria una coordinación entre *CPU* y *GPU* al calcular cada *frame* y operaciones matriciales en ambos lados para crear movimiento.
- Una biblioteca que maneje errores de forma centralizada, pues podría ser realmente caótico y repetitivo de otra manera. Aquí se reunirían mensajes y códigos de error, llamándose cuando quiera comprobarse si un valor es erróneo. Deberá llamarse después de llamar a una función crítica, especialmente.

Desarrollo de la animación Habiendo estudiado el funcionamiento de *OpenGL* y construido un entorno cómodo de trabajo, puede comenzarse el desarrollo del sistema de animación, haciendo uso de la biblioteca propia creada previamente, que facilitará su creación.

El objetivo es construir la animación fotograma por fotograma. Funcionará como una máquina de estados: parado, movimiento e inercia. Inicia en movimiento, entra en inercia por un tiempo y progresa hacia el siguiente elemento en la secuencia. Cuando llegue al final, parará.

- Los parámetros serán inicializados y guardados en una estructura de datos, cuyos valores serán usados y modificados por las funciones.
- Podrán indicarse la secuencia de movimientos de varias formas: indicando qué matrices usar para las transformaciones de forma explícita o mediante funciones de la biblioteca propia, aislando las matrices del funcionamiento.
- También, podrá almacenarse la secuencia cargando desde un *buffer* del usuario o, mediante funciones proporcionadas por la biblioteca, añadir elementos a la secuencia, aislando la memoria del funcionamiento.

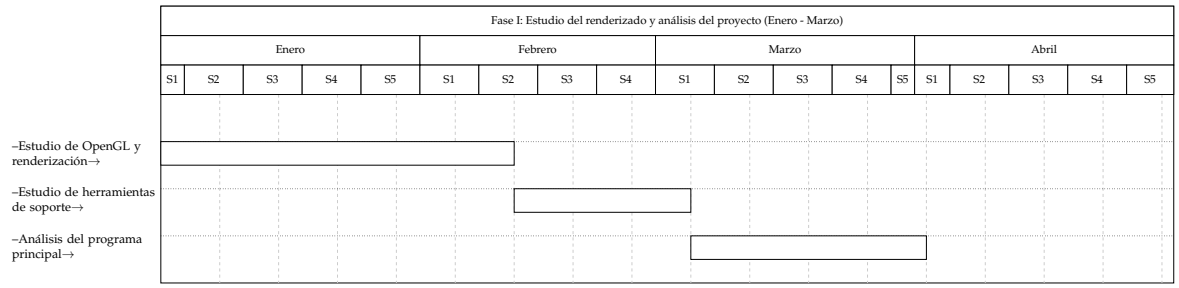
En resumen, se dará libertad al usuario para programar la animación, desde el funcionamiento más manual, aunque más eficiente, a uno más sencillo, aunque menos eficiente.

2.2.3. Fase 3: Pruebas y optimización

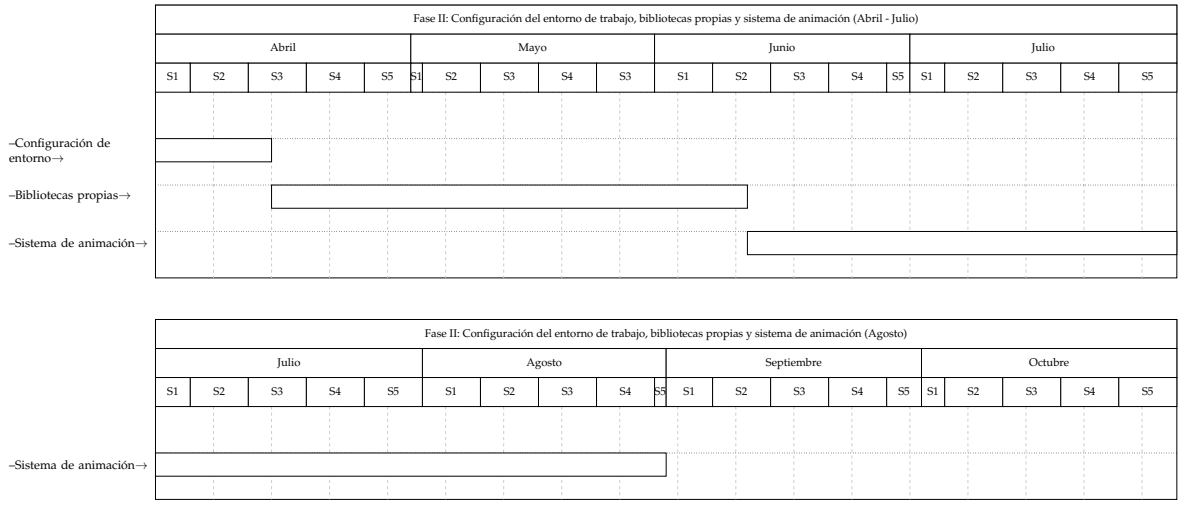
En esta parte, se pondrán a prueba los algoritmos, comparando el rendimiento, uso de memoria y de *CPU* en distintos escenarios. Una vez obtenidos los resultados, observarlos y revisar de nuevo el sistema para detectar posibles mejoras con el objetivo de optimizarlos al máximo.

2.3. Diagrama de Gantt

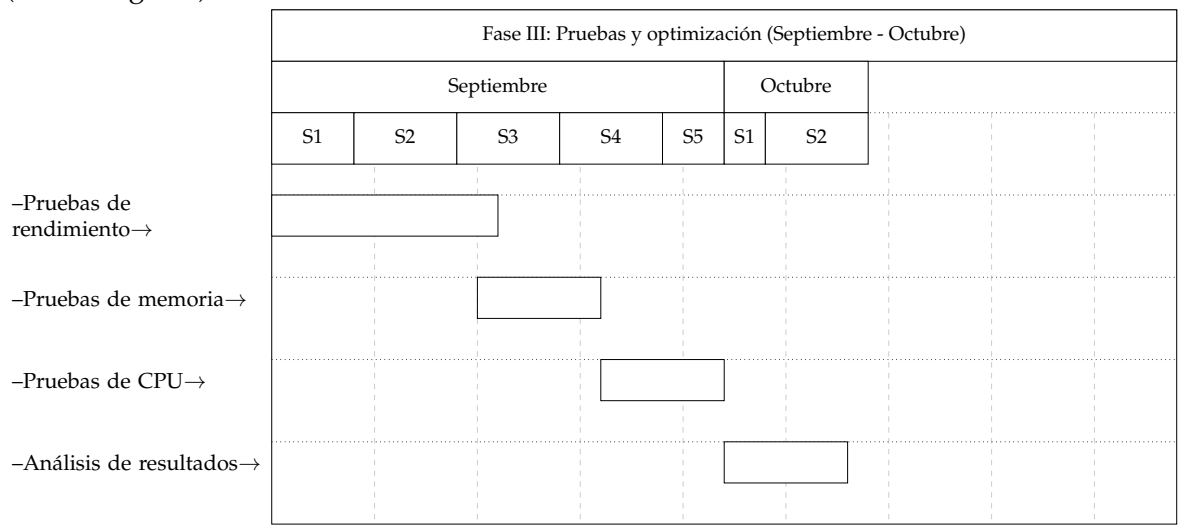
Finalmente, en el diagrama de Gantt de la figura 2.1, se muestra la duración de cada una de las etapas descritas. La fecha de inicio es el 26 de enero y la fecha final es el 13 de octubre. Cada diagrama se divide en **meses** y estos en **semanas**, siendo la unidad de medida los **días**.



(a) Fase I: Estudio del renderizado y análisis del proyecto



(b) Fase II: Configuración del entorno de trabajo, bibliotecas propias y sistema de animación (Abril - Agosto)



(c) Fase III: Pruebas y optimización (Septiembre - Octubre)

Figura 2.1: Diagrama de Gantt detallado del proyecto

3. Marco Conceptual y Estado de la técnica

En este capítulo se hará hincapie en conceptos propios de la renderización *2D*, tales como el *antialiasing* 3.1, y en el estado de la técnica existente: *APIs* más utilizadas 3.2 y software que los implementa 3.3, así como las características más distintivas de estos motores, como podría ser la personalización de *shaders* en *Godot* 3.3.1.

3.1. Antialiasing

Aliasing Esta técnica es fundamental para evitar el denominado *aliasing* [17] y conseguir gráficos sin ruido y suaves. El *aliasing* es un término acuñado del procesamiento de señales, que ocurre en la digitalización de las señales analógicas en las que la frecuencia de muestreo es inadecuada; esta frecuencia es llamada *sampling rate*. Si se analiza la señal con una frecuencia baja, el resultado difiere gravemente de la señal original y resulta en cambios de color abruptos y erróneos. El método para evitar esta distorsión es evidente: aumentar el *sampling rate*; aunque evidente, su coste es alto.

Sampling rate En el contexto de gráficos y renderizado 2D, las formas primitivas serían el mundo analógico descrito matemáticamente que se representan digitalmente en píxeles discretos en una pantalla. Crea el mismo efecto, pues existe un elemento analógico, siendo aquí una línea descrita matemáticamente, y un elemento digital, siendo este un píxel discreto. El *sampling rate* en este caso sería la cantidad de muestras que se toman en cuenta para cada píxel, y cuantas más muestras se tomen, el color final será más cercano al ideal matemático del que se parte. Una manera fácil de entenderlo es imaginar una línea que pasa por un conjunto de cuadrados diagonalmente y entender que se formaría una cantidad abundante de esquinas, en forma de escalera, que dejarían la geometría abrupta y rugosa.

Supersampling Una técnica simple y efectiva es el *supersampling*. Consiste en renderizar la imagen a una mayor resolución para que cada pixel obtenga sus datos de múltiples *sample* . De esta forma, algunos píxeles que antes no se tomarían en cuenta ahora se procesan y se suaviza la forma; el color obtenido por estos píxeles cuyas muestras no son completamente cubiertas se verá alterado y difuminado para conseguir este efecto de suavidad, pues el píxel toma el color promedio de sus *sample* . Es una técnica cara en tiempo de ejecución, aunque funciona.

Multisampling Esta idea se puede expandir y mejorar: *multisampling*. Es una modificación del *supersampling* en la que la idea es la misma, aunque la frecuencia con la que se ejecuta el *fragment shader*, o *FS*, es menor que el número de *samples* por píxel; es común un ratio 4:1.

En la figura 3.1 se compara un mismo modelo con y sin *antialiasing*.

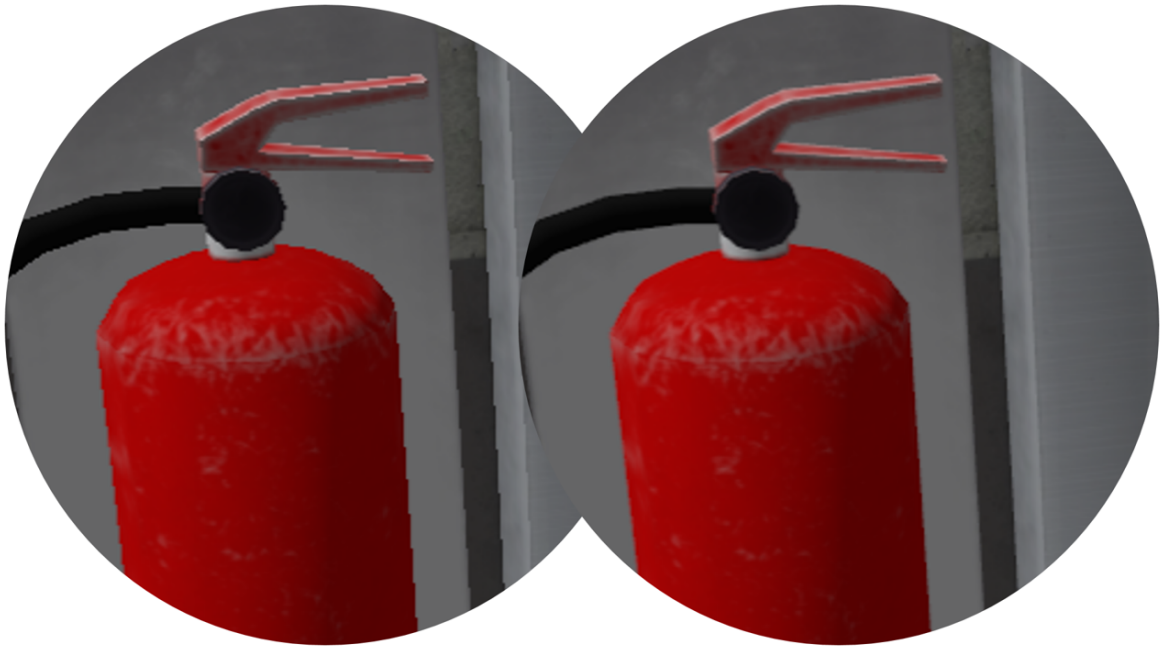


Figura 3.1: Comparación de dos modelos, uno sin antialising y otro con antialiasing.

En la figura 3.2 puede observarse cómo se evalúan los píxeles de un simple triángulo, quedando unos "escalones" muy prominentes. En la figura 3.3 se utiliza *antialiasing MSAA*, en la que se suavizan los bordes alterando el color ligeramente.

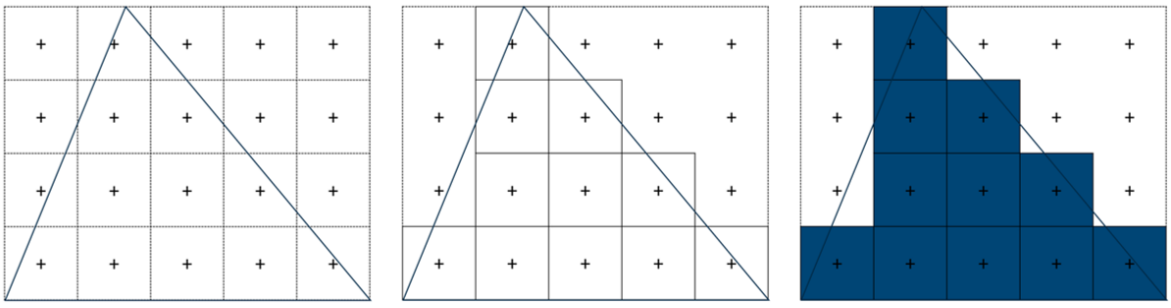


Figura 3.2: Colores de un triángulo siendo evaluados sin antialiasing.

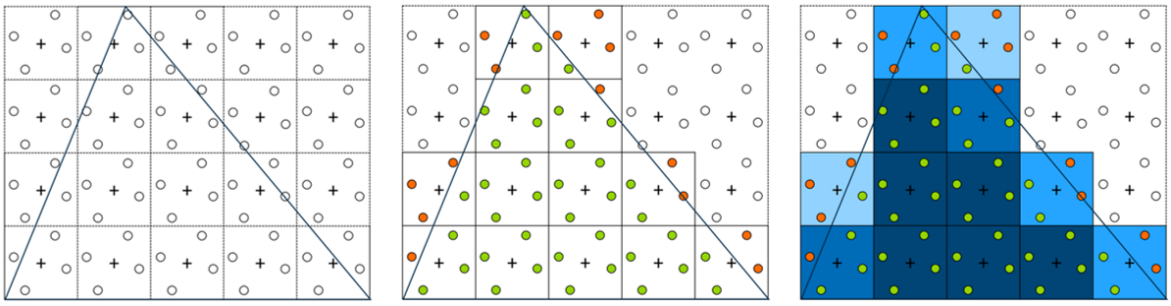


Figura 3.3: Colores de un triángulo siendo evaluados con antialiasing MSAA

3.2. APIs de Renderización

Entre las APIs de renderizado, destacan *OpenGL* [3] y *Vulkan* [3], ambas desarrolladas por *Khronos* [18]. De hecho, *OpenGL* es la predecesora de *Vulkan* y se mantiene como una opción estable y sencilla de utilizar. Aunque *Vulkan* es mucho más flexible, introduce complejidad; concretamente, está dirigido a aplicaciones muy demandantes que necesitan maximizar la eficiencia [19].

3.3. Implementaciones existentes

Antes de abordar el desarrollo de un renderizador, ha sido necesario documentar sobre el funcionamiento de otros motores existentes, de forma que, aunque el propósito sea diferente, el análisis se ha centrado en las partes relacionadas con el proyecto. Cabe destacar que algunos motores, como *Unity* o *Unreal Engine*, son de software privativo y no pueden diseccionarse al detalle como podría hacerse con *Godot*; en este caso, la única documentación disponible es el manual de usuario.

3.3.1. Godot

Renderización 2D [20] y 3D [21] acompañado de un motor de física. Incluye *tilemaps*, partículas y animaciones. Se centra en luces, sombras, movimiento de partículas y visualización de *meshes*. Usa *OpenGL*, *DirectX* y *Vulkan* como *APIs* de gráficos. Usa optimización *GPU*.

Luces y sombras Al igual que el resto del motor, las mecánicas de iluminación consisten en **nodos** o **vértices** en un grafo. Por ejemplo, *DirectionalLight2D* [22] es usado para modelar la luz del sol o de la luna, *CanvasModulate* [23] para dar color a la luz y, el elemento más común, *PointLight2D* [24], que es un punto de luz que podría ser una antorcha, fuego, proyectiles o similar. Para modelar sombra, se puede usar *LightOccluder2D* [25], indicando al *shader* qué parte de la escena debe contener sombras.

2D Meshes Aunque menos usadas que los modelos *3D*, dado que las imágenes son usadas con más frecuencia, *Godot* da completo soporte a los modelos bidimensionales. También, es posible visualizar modelos tridimensionales a través de algunas herramientas en una escena de dos dimensiones. Permite convertir imágenes *Sprite2D* [26] a modelos 2D *MeshInstance2D* [27], ajustando parámetros según la calidad y rapidez que se requiera; puede ser mejor renderizar un modelo a partir de una imagen que directamente visualizar la imagen, especialmente con imágenes grandes, dibujando sólo las partes opacas e ignorando el resto

Animación 2D Hay dos formas de crear animación en dos dimensiones: a través de *AnimatedSprite2D* [28], una manera sencilla que sólo involucra cambio de textura, y *AnimationPlayer* [29], una manera más compleja que permite realizar más acciones en la animación, como reproducir sonidos, activar y desactivar *hitboxes* [30], y demás. Se suele usar una plantilla de imágenes llamada *sprite sheet* que recopila la animación en una sóla imagen, en vez de crear imágenes individuales para cada estado.

Antialiasing 2D Ofrece ajustes sobre el *antialising* generales. Para las líneas puede activarse sin necesidad de *MSAA*. Si se quiere utilizar el efecto sobre cualquier otra geometría, es necesario activar *MSAA 2D* [31] en los ajustes del proyecto.

Shaders *Godot* permite el uso de *shaders* personalizados, con un lenguaje semejante [32] a *GLSL*, el lenguaje de *OpenGL*. Los *shaders* deben estar vinculados a un *Material* [33] y este a un *objeto*. Para interactuar con el *shader* desde el código, se usan *uniform*, de forma semejante a *GLSL*, excepto que las funciones son propias del motor, como es *set_shader_parameter*, indicando simplemente el nombre y el valor a actualizar, y *get_shader_parameter* que recupera el valor de una variable indicando el nombre. Ambas funciones se definen en la clase *Material*.

3.3.2. Blender

Dispone de renderización 2D y 3D. Tiene tres renderizadores, aunque el más relevante es *Cycles* [34]. Basado en físicas, con objetivo de ser fotorrealista. Permite renderización *multi-core* con *CPU* y optimización *SIMD*, a la vez que permite renderización mediante *GPU* y *multi-GPU*. La documentación no es particularmente extensa, aunque el motor es poderoso y cuenta con numerosas características [35], como pueden ser la iluminación, las sombras, las texturas y la perspectiva de cámara.

3.3.3. GTK y GSK

Apuesta por un renderer 2D unificado para *OpenGL* y *Vulkan*, que son compilados de la misma fuente, aunque este cambio es reciente. Previamente,

existía un renderer 2D *OpenGL* que usaban shaders simples aunque más optimizado.

El nuevo renderizador unificado se centra en la mantenibilidad y corrección a largo plazo a cambio de rendimiento. Este lujo es posible gracias a que los programas que hacen uso de este software no son excesivamente demandantes ni requieren una tasa de rendimiento grande, como podrían *Godot*, *Blender*, *Unity* o *Unreal Engine*. También soporta algunas capacidades interesantes, como *antialising*, escalado fraccional y gradientes arbitrarios.

GSK *GTK* [36] aisla los mecanismos de renderización en una dependencia: *GSK* [37]. Con esta abstracción, puede funcionar sin comunicarse directamente con la *API* en cuestión; lo mismo ocurre con los sistemas de ventanas: son abstraídos en una dependencia. *GSK* ofrece una *API* basada en caminos [38], usada para renderizar formas complejas; aunque también se usan para otros cometidos, como trayectorias en animación. El renderizado en *GSK* se basa en **nodos**: clases con diferentes propósitos, construyendo un grafo el cual procesará un tipo de *renderer*, según la *API* gráfica deseada.

3.3.4. Unity

Renderer 3D y 2D que usa *DirectX*, *Metal*, *OpenGL* y *Vulkan*. Utiliza optimización *GPU* para procesar el renderizado. Proporcionan una página web como manual [39], aunque la fuente es software privativo y el código público no está disponible.

3.3.5. Comparación

Existen numerosos sistemas de renderización *2D* y *3D* con diferentes propósitos: videojuegos, cine, modelado, etc. Muchos de ellos son públicos y su funcionamiento puede estudiarse.

La tabla 3.1 compara los motores mencionados: su capacidad dimensional, las *APIs* que soportan, el propósito que tienen y su licencia de uso.

Motor	**2D/3D**	**APIs**	**Objetivo**	**Licencia**
Godot	Ambos	OpenGL, Vulkan y DirectX	Videojuegos: partículas, animaciones, luces y sombras	MIT
Blender	Ambos	CUDA, OptiX, HIP, oneAPI y Metal	Fotorrealismo, basado en físicas	GPL
GTK	2D	OpenGL y Vulkan	Interfaces de usuario	GPL
Unity	Ambos	OpenGL, Vulkan, DirectX y Metal	Videojuegos y animación	Privativa

Tabla 3.1: Comparación de motores de renderización

4. Análisis del problema

En este capítulo se tratarán los requisitos 4.1 se ajustan al proyecto necesarios para el correcto funcionamiento del mismo.

4.1. Requisitos

La tabla de requisitos 4.1 describe 4 tipos: **información** 4.1.1, **funcional** 4.1.2, **no funcional** 4.1.3 y **hardware** 4.1.4. Cada elemento consta de un identificador, un número y una descripción.

Información	**Funcionales**	**No funcionales**	**Hardware**
I.01 Código GLSL	F.01 Biblioteca de shaders	NF.01 Robustez y detección de errores	H.01 Poseer una GPU configurada correctamente
I.02 Resultados, análisis y gráficas	F.02 Biblioteca de errores	NF.02 Modular y organizado	H.02 Sin mínima exigencia de CPU, RAM, disco, etc
	F.03 Biblioteca de polígonos	NF.03 Código comentado y en inglés	
	F.04 Biblioteca miscelánea de C		
	F.05 Biblioteca de animación		
	F.06 Ejemplo de uso y pruebas		

Tabla 4.1: Tabla de requisitos

4.1.1. Requisitos de información

Los requisitos de información describen los documentos y código que debe ser almacenado, leído y procesado.

1. *Shaders*, código *GLSL* externo que necesita ser compilado dentro del programa para renderizar la geometría.

2. Resultados de los programas, almacenados en diferentes formatos con el objeto de llevar a cabo análisis de resultados y transformarlos en gráficas.

4.1.2. Requisitos funcionales

Los requisitos funcionales describen cómo se debe comportar el sistema al ser ejecutado y qué propiedades tiene.

1. Debe existir una biblioteca relacionada con *shaders* que facilite su uso para el usuario.
2. Debe existir una biblioteca de errores que provea al usuario de una manera sencilla de comprobar la validez de ciertas acciones y actuar en consecuencia.
3. Debe existir una biblioteca relacionada con **polígonos**, que construya geometría para el usuario.
4. Debe existir una biblioteca miscelánea que cubra propósitos generales de programación, como leer archivos y crear estructuras de memoria.
5. Debe existir una biblioteca de animación, que soporte crear una secuencia de transformaciones sobre los vértices en pantalla y que simulen movimiento, pudiendo configurar los fotogramas por segundo, acciones y demás.
6. Deben existir programas que utilicen todas las bibliotecas creadas, para servir de ejemplo y sujeto de pruebas para los resultados a analizar.

4.1.3. Requisitos no funcionales

Los requisitos no funcionales describen características más abstractas a las funcionables que describen propiedades ajenas al comportamiento técnico del sistema, como pueden ser la robustez, la modularidad o la fluidez.

1. El sistema debe ser robusto y detectar errores tras operaciones cruciales en la secuencia de los programas.
2. El proyecto debe estar organizado de forma ordenada y modular.
3. El código del proyecto debe estar comentado correctamente, además de estar escrito en el idioma inglés.

4.1.4. Requisitos hardware

1. Poseer una *GPU*, preferiblemente *AMD* o *NVIDIA*, configurada correctamente con los controladores apropiados.
2. No es necesario establecer un requisito mínimo de otros componentes, ya sea *RAM*, *CPU* o disco, pues *OpenGL* es flexible.

5. Lenguaje y Tecnologías Empleadas

Este capitulo aborda conceptos esenciales sobre las tecnologías usadas en el proyecto: lenguajes de programación, *APIs* y herramientas de soporte. Concretamente, se trata el lenguaje *C* 5.1, utilizado durante todo el proyecto, las tecnologías de renderizado 5.2 y las herramientas de soporte utilizadas 5.3.

5.1. Lenguaje C

Puntero Un **Puntero** es una variable que almacena una dirección en memoria de otra variable.

Cabe mencionar que también es posible crear un puntero a una **función**:

```c
#include <stdio.h>
int foo()
{
    return 5;
}

int main()
{
    // Initialize pointer
    int (*fncPtr)();

    // Assign 'foo' function to pointer
    fncPtr = &foo;

    // Implicit call. Some compilers don't support this
    int a = fncPtr();

    // Explicit call
    int b = (*fncPtr)();

    return 0;
}
```

Extracto de código 5.1: Ejemplo de puntero a función en C

Reserva y liberación de memoria En el lenguaje *C*, la gestión de memoria [40] puede ser controlada por el usuario, lo que exige una gran responsabilidad por su parte. Aunque se prefiere siempre que sea posible que el sistema sea el que realice

esta gestión, a veces es necesario: por ejemplo, una función que debe asignar espacio a un *array* con un número de elementos no predefinido. En estos casos, puede hacerse uso de funciones como *malloc* o *calloc*, que inicializan memoria a petición del usuario; si se desea modificar el tamaño de esta memoria, puede utilizarse *realloc* en lugar de una asignación nueva. Una vez terminadas las tareas pertinentes a la asignación de esta memoria solicitada, es necesario liberarla con *free* y, para ser riguroso, darle un valor *NULL* al puntero para evitar posibles confusiones respecto a la disponibilidad del mismo.

5.2. Tecnologías de renderizado

En esta sección se describen, desde el punto de vista de la programación, las *APIs* utilizadas.

Respecto a la sección 5.2.1, que explica los conceptos pertenecientes a *OpenGL* más importantes:

1. En el apartado 5.2.1 se presentan los conceptos necesarios de OpenGL.
2. En 5.2.1 se describe cómo utilizar los *buffers* con el fin de comunicarse con la *GPU*.
3. En 5.2.1 se describe el flujo de *shaders* que recorre una aplicación general que use *OpenGl*.
4. En 5.2.1 se explica el tipo de variable *uniform*, utilizado para comunicarse directamente con valores de *shaders*.
5. En 5.2.1 se describe cómo indicar al *shader* qué valores utilizar y dónde se encuentran dentro de los *buffers*, mencionados previamente.
6. En 5.2.1 y 5.2.1 se describe con ejemplos de código el proceso para crear *shaders* y un programa a partir de éstos.
7. En la sección 5.2.2 se describe el lenguaje *GLSL*, utilizado con el fin de programar *shaders* para *OpenGL*.

5.2.1. OpenGL

Aunque también mencionado en el apartado 3.2, de las API existentes, *OpenGL* y *Vulkan*, en este trabajo se ha decidido usar *OpenGL*, ya que es una *API* muy utilizada y estable, a la par que altamente flexible (aunque no al nivel de *Vulkan*).

Funcionamiento principal Existen dos formas de utilizar *OpenGl*: *Immediate mode* y *core-profile*. El primer modo es más fácil de entender y usar, aunque tiene funciones obsoletas y no se aconseja su uso. La manera recomendada de programar es con *core-profile*, una división de la especificación con funcionalidades modernas.

La especificación de *OpenGL* permite extensiones para que los desarrolladores no necesiten esperar a una nueva versión.

OpenGL se puede entender como una gran máquina de estados, llamándose a cada uno de ellos un **contexto**. Este contexto puede ser modificado por funciones y utilizado por otras. El código está escrito en *C*, aunque permite derivaciones para distintos lenguajes. Se usan **objetos**: cada objeto es un conjunto de opciones que representan un *subset* del estado de *OpenGL*. Por ejemplo, un objeto podría representar el tamaño y colores de una ventana.

Los objetos se crean, se modifican y se utilizan mediante la *API*, y la información se almacena fuera de la vista del programador. Esto permite una flexibilidad y reusabilidad increíble, pues se pueden crear varios objetos y vincularlos para dibujarlos de forma rápida.

El extracto de código 5.2 podría ser (son comandos ficticios) un ejemplo de creación y manipulación de un objeto:

```
// Create the storage for the object
GLuint objectName;
glGenObject(1, &objectName);

// Put data into the object.
glBindObject(GL_MODIFY, objectName);
glObjectParameteri(GL_MODIFY, GL_OBJECT_COUNT, 5);
glObjectParameterf(GL_MODIFY, GL_OBJECT_OPACITY, 0.4f);
glObjectParameters(GL_MODIFY, GL_OBJECT_NAME, "Some String");
```

Extracto de código 5.2: Ejemplo de objeto en OpenGL

Buffers A continuación se detallará la especificación de vértices [41] en *OpenGL*.

- El *Vertex Buffer Object* (VBO) es un*buffer* que puede almacenar una gran cantidad de vértices en memoria. El objetivo es envíar el mayor número de datos a la vez posible.
- Un *Vertex Array Object* (VAO) es un *buffer* enlazable, como es el *VBO*. Almacena la configuración de los atributos de los vértices. Está fuertemente vinculado al *VBO*, ya que el *VAO* almacena la configuración de los vértices almacenados en él. El *VAO* almacena las llamadas a atributos y vértices, por lo que para dibujar, una vez configurado, sería necesario simplemente vincularlo. Cabe mencionar que, si la configuración o el *VBO* cambiase, sería necesario actualizar el *VAO*.
- Un *Element Buffer Object* (EBO), también llamado *Index Buffer*, permite almacenar en él indices que referencian vértices en el *VBO*. En vez de duplicar vértices, pueden referenciarse múltiples veces. Por ejemplo, un cuadrado contiene 2 triángulos y 6 vértices. Utilizando un *EBO*, podrían declararse 4 vértices e indicar el orden en el que se utilizan para crear los triángulos.

En el ejemplo 5.3 se muestra un ejemplo de creación de los *buffers VBO* y *VAO*. Una vez creados y configurados, en la función *display* ha de vincularse el *VAO* para cargar los vértices y su configuración, como en el ejemplo 5.4.

```
/*
 * Initialize data related to OpenGL functions
 */
void
initGLData()
{
    GLfloat triangle[3][2] = {
        {1, -1},
        {0, 1},
        {-1, -1}
    };

    // Create VAO
    glGenVertexArrays(1, &vao);

    // Bind VAO
    glBindVertexArray(vao);

    // Create VBO
    glGenBuffers(1, &vbo);

    // Bind VBO
    glBindBuffer(GL_ARRAY_BUFFER, vbo);

    // Copy vertices data to the VBO
    glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 6, triangle,
    GL_STATIC_DRAW);

    // Specify attribute positions
    glVertexAttribPointer(vPosition, 2, GL_FLOAT, GL_FALSE, 0, (void*)0);

    // Enable the vPosition attribute
    glEnableVertexAttribArray(vPosition);
}
```

Extracto de código 5.3: Ejemplo de creación de los objetos VBO y VAO.

```
// Display function for the GLUT loop
void
display()
{
    glClearColor(0.7421875, 0.33203125, 0.015625, 1);
    glClear(GL_COLOR_BUFFER_BIT);

    // Bind the VAO and load vertices and configuration
    glBindVertexArray(vao);

    glDrawArrays(GL_TRIANGLES, 0, 3);
    glFlush();
}
```

Extracto de código 5.4: Ejemplo de uso de los objetos VBO y VAO.

El *EBO* puede usarse para dibujar un cuadrado. Se debe añadir después de vincular el *VAO*, dejando el resto de la misma manera. La función de *display* es idéntica, aunque se sustituye *glDrawArrays* por *glDrawElements*. El ejemplo 5.5 se muestra la creación de este *buffer EBO* y en el ejemplo 5.6 el uso de éste.

```
float vertices[] = {
     0.5f,  0.5f, 0.0f,  // top right
     0.5f, -0.5f, 0.0f,  // down right
    -0.5f, -0.5f, 0.0f,  // down left
    -0.5f,  0.5f, 0.0f   // top left
};

unsigned int indices[] = {
    0, 1, 3,   // First triangle
    1, 2, 3    // Second triangle
};

// Create EBO
glGenBuffers(1, &ebo);

// Bind EBO
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, ebo);
glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices,
   GL_STATIC_DRAW);
```

Extracto de código 5.5: Ejemplo de creación de EBO.

```
// Display function for the GLUT loop
void
display()
{
    glClearColor(0.7421875, 0.33203125, 0.015625, 1);
    glClear(GL_COLOR_BUFFER_BIT);

    // Bind the VAO and load vertices and configuration
    glBindVertexArray(vao);

    // Draw triangles by the indices
    glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0);

    glFlush();
}
```

Extracto de código 5.6: Ejemplo de uso de EBO.

Flujo de shaders El flujo de shaders [42] de *OpenGL* puede dividirse en 6 etapas fundamentales: *Vertex Shader*, *Tesselation*, *Geometry Shader*, *Primitive Assembly*, *Rasterization*, *Fragment Shader* y *Tests and Blending*. Hay algunas fases en las que el programador puede introducir su propia implementación, como se mostrará más adelante.

Se usa un lenguaje especial de *OpenGL* para *shaders*, llamado *GLSL*, para compilar y generar estos *shaders*. Este lenguaje se tratará más adelante.

1. **Vertex Shader**. Se transforman vectores de posición de los vértices en otros diferentes, generalmente 3D.
2. **Tessellation**. Se subdivide la geometría compleja en Primitivos más simples.
3. **Geometry Shader**. Se modifican, añaden o eliminan formas. Por ejemplo, romper un triángulo en tres, eliminar formas que no necesita o cambiar un punto en un cuadrado.
4. **Primitive Assembly**. Se toman los vértices procesados y se agrupan en formas simples, como puntos, líneas o triángulos.
5. **Rasterization**. Se transforman formas 3D en piezas 2D llamadas fragmentos (potenciales píxeles). En esta fase se determina qué píxeles en la pantalla cubre cada forma y qué datos lleva cada fragmento.
6. **Fragment Shader**. En este paso se decide cómo se verá cada fragmento, es decir, color, luces, sombras, textura, etc.
7. **Tests and Blending** (Per-Sample Operations). En este paso final se decide qué fragmentos finalmente ocuparán píxeles en la pantalla. Se hacen pruebas como el *Depth Test*, el *Stencil Test* o el *Scissor Test*. Después de estas pruebas se crea la imagen final con los fragmentos restantes.

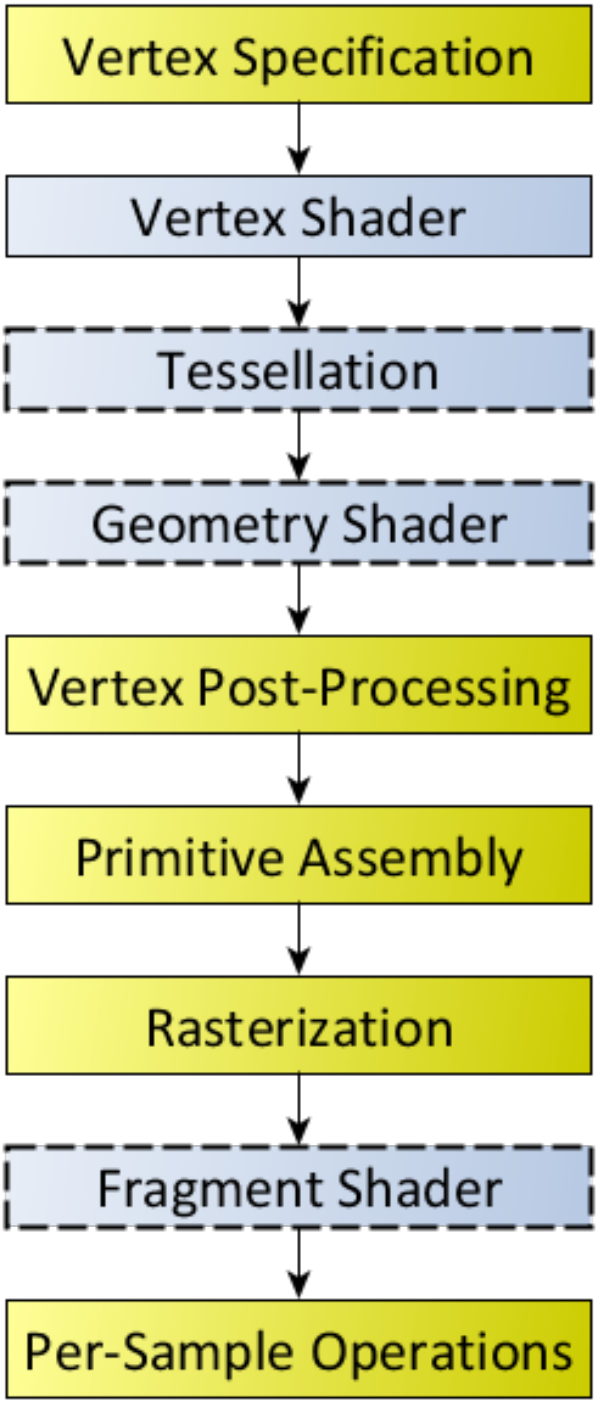


Figura 5.1: Diagrama de Khronos que describe el flujo de shaders en OpenGL.

Uniforms Los *uniforms* son una manera de enviar datos de la aplicación en *CPU* a los *shaders* en *GPU*. Son variables globales y pueden ser accedidas por cualquier *shader*. Se añade la palabra clave *uniform* a la variable en el *shader* para convertirla en *uniform*. Esta variable debe ser declarada en el *shader*, de la misma forma que cualquier variable en *GLSL*, aunque con el prefijo *uniform*. Para acceder a ella desde el código *C*, primero se debe localizar el atributo y entonces, dar un valor a la variable. En la figura 5.7 se muestra un ejemplo.

```
// Getting time value
float timeValue = glfwGetTime();

// Computing green colour with sin function
float greenValue = (sin(timeValue) / 2.0f) + 0.5f;

// Locating uniform attribute
int vertexColorLocation = glGetUniformLocation(shaderProgram, "ourColor")
    ;

// Activating program
glUseProgram(shaderProgram);

// Inserting computed value into the uniform value
glUniform4f(vertexColorLocation, 0.0f, greenValue, 0.0f, 1.0f);
```

Extracto de código 5.7: Ejemplo de uso de variables Uniform.

Atributos de vértices Para indicarle a *OpenGL* cómo utilizar los datos que ha recibido, es necesario indicarlo explícitamente. Por ejemplo, si los vértices tienen 3 coordenadas, siendo cada una de ellas float, que usualmente son 4 bytes, esto significa que cada vértice serán 12 bytes, y si el polígono deseado fuera un triángulo, serían 3 vértices: 36 bytes. Se debe tener en cuenta que no hay espacio entre los valores en este caso. Un ejemplo de configuración de atributos sería la figura 5.8.

Los argumentos de *glVertexAttribPointer* son:

1. **Índice**. Posición del atributo. Puede definirse en el shader o conseguirse dinámicamente.
2. **Tamaño** del atributo.
3. **Tipo** de dato. Las coordenadas se guardan como *float*.
4. **Normalización**. En este caso, las coordenadas están ya normalizadas previamente, por lo que se deja en falso.
5. **Stride**. Es el espacio entre atributos consecutivos. Si es *0*, se asumirá que los datos siguen uno detrás del otro sin espacio entre ellos.

```
GLfloat triangle[3][2] = {
    {1, -1},
    {0, 1},
    {-1, -1}
};

...

// Copy vertices data to the VBO
glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 6, triangle,
    GL_STATIC_DRAW);

// Specify attribute positions
glVertexAttribPointer(vPosition, 2, GL_FLOAT, GL_FALSE, 0, (void*)0);

// Enabling attribute
glEnableVertexAttribArray(vPosition);
```

Extracto de código 5.8: Ejemplo de configuración de atributos.

6. **Offset**. Es donde empieza la posición de los datos en el *buffer*. Requiere un puntero a *void*.

Compilación de un shader Con el propósito de mantener la sencillez en el ejemplo, el código *GLSL* se almacenará en constantes dentro del código *C*, el cual se le proporciona a *OpenGL* y se compila; en el ejemplo 5.9 se muestra este concepto.

```
// Shader object is created and stores reference
unsigned int vertexShader, fragmentShader;
vertexShader = glCreateShader(GL_VERTEX_SHADER);
fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);

// Linking code to the object
glShaderSource(vertexShader, 1, &vertexShaderSource, NULL);
glCompileShader(vertexShader);
glShaderSource(fragmentShader, 1, &fragmentShaderSource, NULL);
glCompileShader(fragmentShader);

// Checking errors
int success;
char infoLog[512];

glGetShaderiv(vertexShader, GL_COMPILE_STATUS, &success);
if(!success)
{
    glGetShaderInfoLog(vertexShader, 512, NULL, infoLog);
    std::cout << "ERROR::SHADER::VERTEX::COMPILATION_FAILED\n" << infoLog
    << std::endl;
}

glGetShaderiv(fragmentShader, GL_COMPILE_STATUS, &success);
if(!success)
{
    glGetShaderInfoLog(fragmentShader, 512, NULL, infoLog);
    std::cout << "ERROR::SHADER::FRAGMENT::COMPILATION_FAILED\n" <<
   infoLog << std::endl;
}
```

Extracto de código 5.9: Ejemplo de compilación de un shader

Creación de un programa Una vez se tienen los *shaders* compilados por separado, es necesario enlazarlos para combinarlos. Esto se hace mediante un **programa**. Se vinculan los shaders a este programa, que se activa cuando se renderizan objetos. Las salidas de cada *shader*, entonces, se vinculan a las entradas del siguiente. El código necesario para la creación del programa podría ser como el de la figura 5.10.

```
// Program object creation
unsigned int shaderProgram;
shaderProgram = glCreateProgram();

// Enlace de los Shaders al programa
glAttachShader(shaderProgram, vertexShader);
glAttachShader(shaderProgram, fragmentShader);
glLinkProgram(shaderProgram);

// Error checking
glGetProgramiv(shaderProgram, GL_LINK_STATUS, &success);
if(!success)
{
    glGetProgramInfoLog(shaderProgram, 512, NULL, infoLog);
    std::cout << "ERROR::SHADER::PROGRAM::LINKAGE_FAILED\n" << infoLog <<
    std::endl;
}

// Program activation
glUseProgram(shaderProgram);

// Cleaning of used shaders
glDeleteShader(vertexShader);
glDeleteShader(fragmentShader);
```

Extracto de código 5.10: Ejemplo de creación de un programa

5.2.2. GLSL

Los shaders deben ser compilados en tiempo de ejecución. Aunque no se haya profundizado en el lenguaje *GLSL* [43], es sencillo de entender una vez aprendido *C*.

El código de la figura 5.11 es un ejemplo muy básico de un *Vertex Shader* que simplemente pasa las coordenadas al *shader*, sin manipularlas:

La primera línea indica la versión que se utiliza, que es semejante a la versión de *OpenGL* que se esté usando (3.3). La segunda línea indica la posición en la que se encuentra el dato *aPos*, que es un vector de 3 dimensiones. Este sería el vector que indica la posición de un vértice. La posición se transporta a un vector de 4

```
version 330 core
layout (location = 0) in vec3 aPos;

void main()
{
    gl_Position = vec4(aPos.x, aPos.y, aPos.z, 1.0);
}
```

Extracto de código 5.11: Ejemplo de Vertex Shader

dimensiones, en la que la última coordenada es un dato llamado *perspective division*, en la función principal del *shader*.

El código de la figura 5.12 es un ejemplo muy básico de un *Fragment Shader* que hace que todos los colores sean el naranja.

```
#version 330 core
out vec4 FragColor;

void main()
{
    FragColor = vec4(1.0f, 0.5f, 0.2f, 1.0f);
}
```

Extracto de código 5.12: Ejemplo de Fragment Shader

La instrucción out hace que devuelva una variable: *FragColor*, que decide el color final. En este caso, es un vector de 4 dimensiones, que indica los tres colores primarios: rojo, verde y azul (RGB) y el valor alpha (opacidad).

5.3. Herramientas de soporte

En esta sección se tratarán herramientas de soporte, útiles para la gestión y organización del proyecto, tanto en la estructura como en el código, tales como *CMake* 5.3.1, que gestiona la compilación y dependencias, *GLEW* 5.3.2, que elimina la necesidad de obtener las funciones de *OpenGL* manualmente, *Freeglut* 5.3.3, que gestiona la creación de ventanas, o *Git* 5.3.4, que gestiona el control de versiones.

5.3.1. CMake

Se usará *CMake* [16] para gestionar dependencias y compilación del proyecto.

CMake maneja dependencias de forma estable y es beneficioso a largo plazo, cuando el código empieza a aumentar de tamaño y comienza a ser insostenible compilar a mano. Otra alternativa sería manualmente escribir *Makefile*. Se consideró más útil y profesional usar *CMake*, ya que tiene un nivel de abstracción sobre los *Makefile* muy pulido y efectivo que costaría replicar a mano.

5.3.2. GLEW

Las funciones de *OpenGL* se cargan de maneras diferentes según la plataforma en la que nos encontremos. En *GNU/Linux*, bajo la implementación de *Mesa* [44], están dinámicamente enlazadas, por lo que hay que recuperarlas de memoria. En otras plataformas es distinto, por lo que *GLEW* es muy útil para resolver este problema.

GLEW [45] inicializa las funciones de *OpenGL* de forma agnóstica al sistema operativo. Esto libera al programador de una tediosa carga, pues hacerlo de forma

manual sería fatigador y la mejora en rendimiento sería mínima o nula. El código necesario para la inicialización de *GLEW* es sencillo y se muestra en la figura 5.13.

```
// Initializing OpenGL functions with GLEW

GLenum err = glewInit();
if (GLEW_OK != err)
{
    /* Problem: glewInit failed, something is seriously wrong. */
    printf("Error: %s\n", glewGetErrorString(err));
}
printf("Status: Using GLEW %s\n", glewGetString(GLEW_VERSION));
```

Extracto de código 5.13: Ejemplo de inicialización de GLEW

5.3.3. Freeglut

Freeglut [46] nace como alternativa a *GLUT* [47], una biblioteca ya abandonada. Se encarga de todas las tareas específicas al sistema operativo necesarias para crear ventanas, inicializar contextos de *OpenGL*, gestionar eventos de entrada y permite crear programas realmente portables con *OpenGL*. El codigo necesario para inicializar *Freeglut* podría ser como en la figura 5.14:

```
// Calling glutInit before all other glut functions
glutInit(&argc, argv);

// Setting up glut variables
glutInitDisplayMode(GLUT_RGBA);
glutInitWindowSize(512, 512);
glutInitContextVersion(4,3);
glutInitContextProfile(GLUT_CORE_PROFILE);
glutCreateWindow(argv[0]);

// Initializing OpenGL functions with GLEW
GLenum err = glewInit();
if (GLEW_OK != err)
{
    /* Problem: glewInit failed, something is seriously wrong. */
    printf("Error: %s\n", glewGetErrorString(err));
}
printf("Status: Using GLEW %s\n", glewGetString(GLEW_VERSION));

// GLUT main loop, no return
glutMainLoop();
```

Extracto de código 5.14: Ejemplo de inicialización de Freeglut

5.3.4. Git

La tecnología *Git* se ha utilizado durante todo el proyecto como software de **control de versiones**, haciendo gran uso de las ramas para distintas funcionalidades, las cuales evitan un posible colapso irremediable permitiendo deshacer cambios rápidamente.

6. Diseño del sistema

En este capítulo se detalla el diseño del sistema, especificando a alto nivel cada parte que lo compone, incluyendo, también, esquemas que ilustran su estructura. Concretamente, se describirán las bibliotecas de ***shaders*** 6.2, **errores** 6.3, código **misceláneo** *C* 6.4, **polígonos** 6.5 y **animaciones** 6.6. En cada una de estas secciones, se tratan las decisiones de diseño tomadas respecto a cada biblioteca donde se expone cada elemento de la biblioteca y unas directrices sobre cómo utilizarlas desde la perspectiva del usuario.

6.1. Leyenda

La figura 6.2 es la leyenda que representa los elementos con los que se describe el sistema en los próximos diagramas de bibliotecas.

A continuación, se describen los diferentes componentes de sistema acompañados de diagramas que indican los elementos que lo componen y su relación:

- **Lista azul**: indica una estructura de datos.
- **Lista verde**: indica un tipo **enumerado**.
- **Rectángulo**: indica una función o *macro*
- **Flechas**: indican la dependencia entre elementos, de forma que el elemento origen es la dependencia del elemento destino.

En los sucesivos apartados, se describen cada uno de los módulos que componen el sistema, comenzando por la biblioteca de *shaders*. La figura 6.1 muestra la estructura general de las bibliotecas empleadas en el proyecto y las relaciones de dependencia entre ellas.

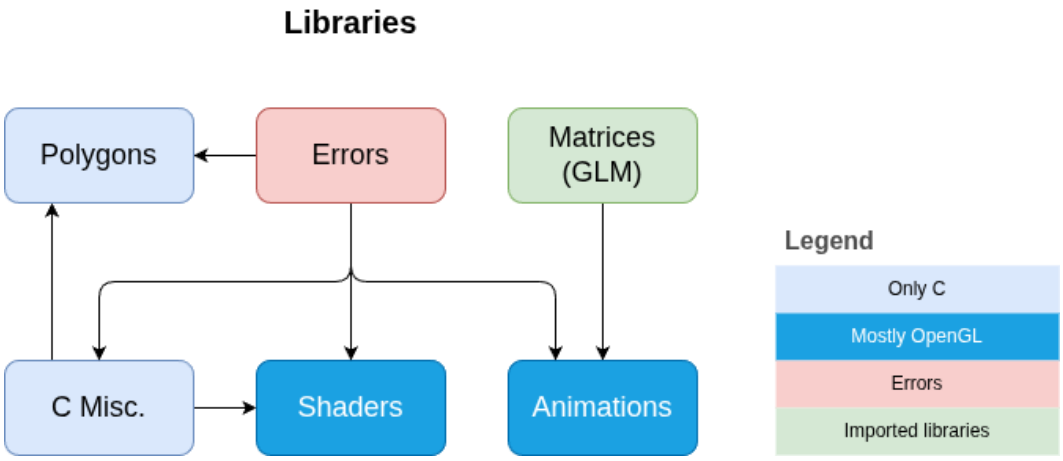


Figura 6.1: Diagrama general de las bibliotecas y sus relaciones.

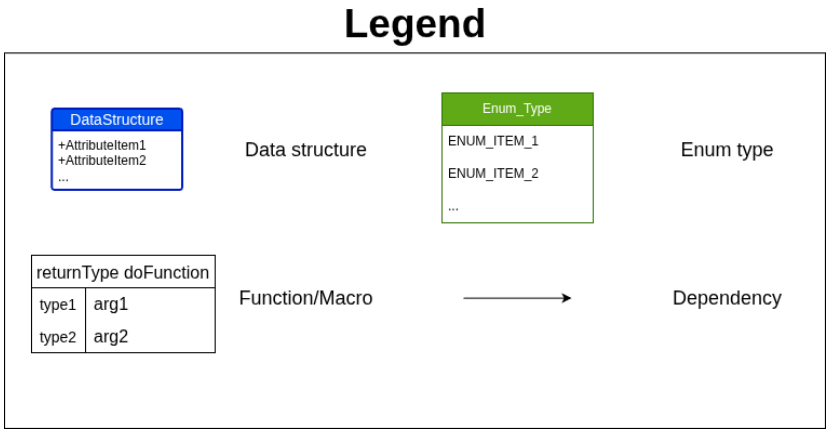


Figura 6.2: Leyenda de los diagramas de las bibliotecas.

6.2. Shaders

En esta sección se trata la biblioteca de *shaders*: su motivación, funciones, estructuras de datos y la *API* pública que ofrece. En la figura 6.3 se representa la estructura de la biblioteca de *shaders*, la cual existe con el propósito de facilitar el procesamiento de los mismos, en lugar de manualmente repetir el proceso cada vez que se deseen usar. Así, no es necesario leer, compilar y enlazar los *shaders* manualmente.

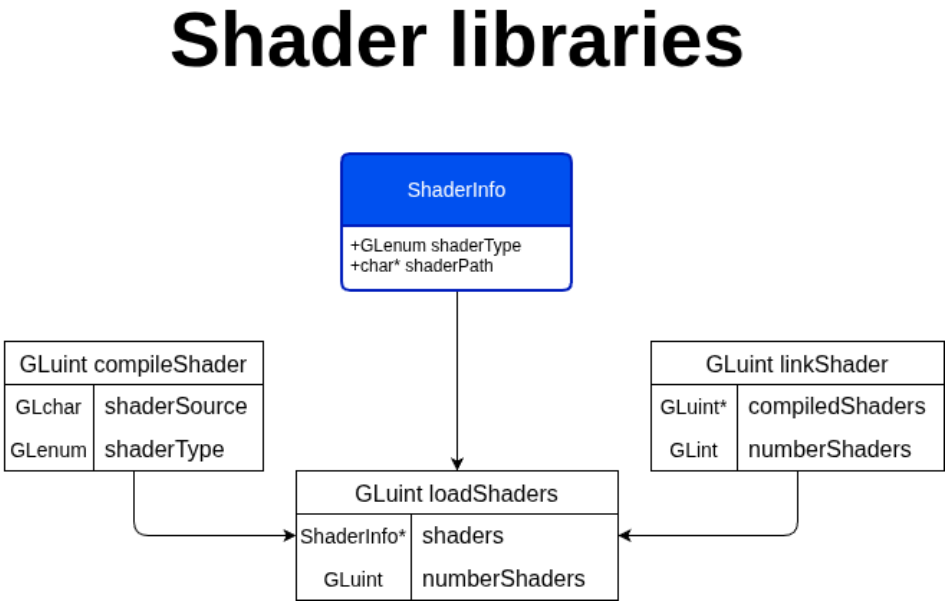


Figura 6.3: Diagrama de la biblioteca de shaders.

6.2.1. Decisiones de diseño

```
1  typedef struct ShaderInfoStruct
2  {
3      const GLenum shaderType;
4      const char* shaderPath;
5  } ShaderInfo;
6
7  GLuint compileShader(const GLchar* shaderSource, GLenum shaderType);
8  GLuint linkShader(GLuint* compiledShaders, GLuint size);
9
10 GLuint loadShaders(ShaderInfo* shaders, GLuint numberShaders);
```

Extracto de código 6.1: Prototipos de la biblioteca de shaders

El código 6.1 muestra el prototipo de la biblioteca en código *C*.

- La estructura *ShaderInfo* contiene el tipo del *shader* y una cadena de caracteres contenido la ruta hacia el shader.
- Se ha separado el proceso del *shader* en dos funciones que los compilan y enlazan en un programa: *compileShader* y *linkShader*. Luego, se utilizan las dos juntas en la función *loadShader* junto a los datos recibidos en la estructura *ShaderInfo*. De esta manera, en lugar de una sóla función con muchos argumentos que realice muchas tareas, se divide en varias partes y cooperan entre ellas.
    1. La función *compileShader* toma el contenido y el tipo del *shader* para luego compilarlo. Devuelve su identificador como número entero sin signo.
    2. La función *linkShader* toma una lista de *shaders* compilados, es decir, sus identificadores, y los enlaza creando un programa. Devuelve su identificador como número entero sin signo.
    3. La función *loadShaders* toma una lista de *ShaderInfo* y los procesa utilizando las dos funciones anteriores, generando un programa. Para cada *shader*, lee su contenido del fichero en la ruta correspondiente y lo compila según el tipo; los dos datos proporcionados por *ShaderInfo*. Una vez compilados todos, se enlazan en un sólo programa. Devuelve su identificador como número entero sin signo.

6.2.2. API pública

El usuario deberá utilizar *ShaderInfo*, completándola con los datos del tipo del *shader* y su contenido en forma de cadena de caracteres. Posteriormente, llamar a la función *loadShader* recibiendo estos datos; aunque se deja libertad para realizar el proceso manualmente, pudiendo llamar por separado a *compileShader* y *linkShader*.

- Funciones internas:
    - *compileShader*.
    - *linkShader*.
- Funciones públicas:
    - *loadShader*.
- Estructuras públicas:
    - *ShaderInfo*.

6.3. Errores

En esta sección se trata la biblioteca de **errores**: su motivación, funciones, estructuras de datos y la *API* pública que ofrece. La figura 6.4 representa la biblioteca de errores, que tiene como objetivo comprobar si el valor recibido es

adecuado y proporcionar mensajes de error, a la vez que devuelve un número que indica si fue válido o no. De esta manera, se centralizan los errores y se reduce la comprobación caótica de ellos en el código. También, sería sencillo determinar qué tipo de error se intenta comprobar al revisar el código.

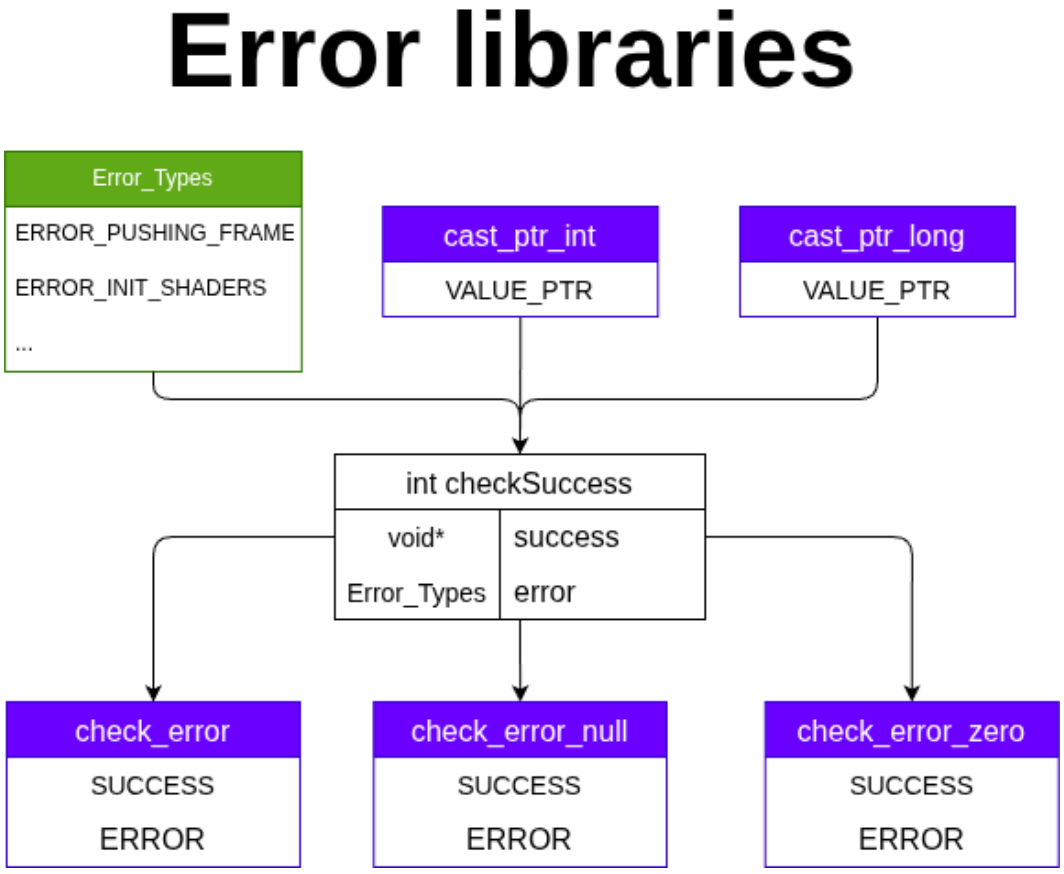


Figura 6.4: Diagrama de la biblioteca de errores.

6.3.1. Decisiones de diseño

El código 6.2 muestra el prototipo de la biblioteca en código *C*.

- Se recopilan los tipos de errores en un enumerado llamado *Error_Types*: se organizan mediante texto y se traducen a números. De esta forma, es fácil distinguir qué error se pretende comprobar sin perder eficiencia.
- La función principal es *checkSuccess*, que comprueba la validez de un valor según un tipo de error. Se utilizan también funciones *macro* auxiliares para convertir tipos y retornar.

    1. La función *checkSuccess* toma una dirección a un valor y el tipo de error que se quiere comprobar, con el que se decide a qué tipo se debe transformar el valor y qué condición evaluar. De esta forma, puede

```c
#define check_error(SUCCESS, ERROR) if (checkSuccess(SUCCESS, ERROR))
    return -1

#define check_error_null(SUCCESS, ERROR) if (checkSuccess(SUCCESS, ERROR)
    ) return NULL

#define check_error_zero(SUCCESS, ERROR) if (checkSuccess(SUCCESS, ERROR)
    ) return 0

#define cast_ptr_int(VALUE_PTR) (*( (int*)VALUE_PTR))

#define cast_ptr_long(VALUE_PTR) (*( (long*)VALUE_PTR))

#define check_null(VALUE) (VALUE == NULL) ? 1 : 0

typedef enum {
    ERROR_PUSH_FRAME,
    ERROR_CREATING_POLYGON,
    ERROR_GENERATING_ANIMATION,
    ERROR_LOADING_ANIMATION,
    ERROR_COPYING_UNIFORM,
    ERROR_EXTENDING_ANIMATION,
    ERROR_ALLOCATING_MEMORY,
    ERROR_REALLOCATING_MEMORY,
    ERROR_COPYING_MEMORY,
    ERROR_INIT_SHADERS,
    ERROR_INIT_GL_DATA,
    ERROR_INIT_ANIMATION_DATA,
    ERROR_LOADING_SHADERS,
    ERROR_OPENGL_CALL,
    ERROR_OPENING_FILE,
    ERROR_SIZE_FILE,
    ERROR_READING_FILE,
    ERROR_SEEK_FILE,
    ERROR_CLOSING_FILE,
    ERROR_CREATING_SHADER,
    ERROR_COMPILING_SHADER,
    ERROR_LINKING_SHADER,
    ERROR_CREATING_PROGRAM,
    ERROR_CREATING_ARRAY,

} Error_Types;

int checkSuccess(void* success, Error_Types error);
```

Extracto de código 6.2: Prototipos de la biblioteca de errores

determinarse qué tipo es el valor original sin crear demasiadas funciones. Por supuesto, conllevaría sus riesgos, concretamente en el mal uso del usuario al enviar los datos. Devuelve 0 en éxito y 1 en fracaso.

2. Las funciones *macro cast_ptr_int* y *cast_ptr_long* toman una dirección a un valor y devuelven ese mismo valor convertido al tipo *int* o *long*, respectivamente.
3. En el caso de que se desee retornar un valor y finalizar la función en caso de que sea inválido, se han diseñado unas funciones *macro* que comprueban la salida de *checkSuccess* y devuelven un valor deseado: *check_error*, *check_error_null*, *check_error_zero*, que devuelven negativo, nulo y cero, respectivamente.

6.3.2. API pública

El usuario debería, tras una función crítica, comprobar el valor, ya sea llamando a alguna de las *macros* o *checkSuccess*, dependiendo de si se quiere devolver un valor o no.

6.4. Misceláneo

En esta sección se detalla la biblioteca de funciones **misceláneas**: su motivación, funciones, estructuras de datos y la *API* pública que ofrece.

En la biblioteca miscelánea de *C* 6.5 se reúnen funciones auxiliares no dirigidas especialmente a *OpenGL* y compuestas por código *C* sin dependencias. Principalmente, se ha diseñado con la intención de ser utilizada para la lectura de ficheros, la creación de estructuras de datos y otras cuestiones generales de programación en *C*.

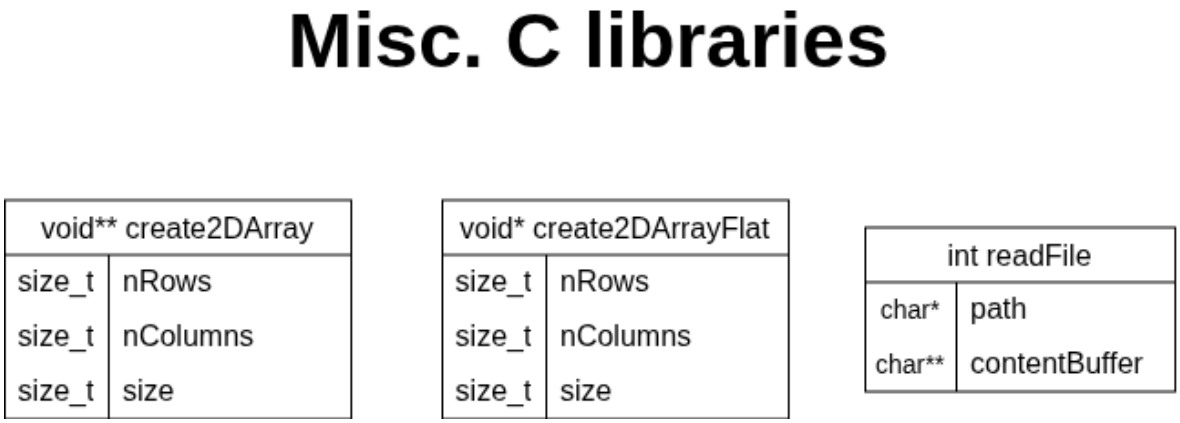


Figura 6.5: Diagrama de la biblioteca de funciones misceláneas de C.

6.4.1. Decisiones de diseño

```
int readFile(const char* path, char** contentBuffer);

void** create2DArray(size_t nRows, size_t nColumns, size_t size);

void* create2DArrayFlat(size_t nRows, size_t nColumns, size_t size);
```

Extracto de código 6.3: Prototipos de la biblioteca miscelánea

El código 6.3 muestra el prototipo de la biblioteca en código *C*.

- Para leer archivos se crea la función *readFile*. Se crean las funciones *create2DArray* y *create2DArrayFlat* para, rápidamente, asignar memoria a una estructura de datos en forma de lista de listas o una sóla lista aplanada.

    1. La función *readFile* toma una ruta hacia un fichero y almacena sus contenidos en un *buffer*. Se realiza la lectura de un fichero cada vez que se procesa un *shader*, por lo que es una función muy conveniente.
    2. Se crean las funciones *create2DArray* y *create2DArrayFlat* para crear una matriz *2D*. La primera función consistiría en crear una lista de listas, mientras que en la segunda está aplanada, es decir, los elementos son consecutivos. Principalmente, están pensadas para utilizarse en la creación de matrices de coordenadas bidimensionales.

6.4.2. API pública

El usuario debería utilizar *readFile* aportando una ruta hacia un fichero y un *buffer* donde asignar memoria y almacenar sus contenidos.

Las restantes dos funciones deberían utilizarse aportando el tamaño de las columnas y filas, además de un *buffer*, semejante al de *readFile*. Entonces, el *buffer*, aunque vacío, tendrá asignada la memoria deseada para una matriz bidimensional.

6.5. Polígonos

En esta sección se trata la biblioteca de **polígonos**: su motivación, funciones, estructuras de datos y la *API* pública que ofrece.

6.5.1. Decisiones de diseño

Esta biblioteca de polígonos 6.8 trata la creación de polígonos. En esta versión es bastante reducida y en futuras versiones se espera un crecimiento en las funcionalidades. La función principal crea un polígono regular de un número de

vértices. En lugar de crear manualmente polígonos vértice a vértice, esta biblioteca sirve al usuario para generarlos automáticamente con algunos parámetros.

- El programa *createNGonVertices* construye un polígono regular de n vértices en el centro de la pantalla, junto a un *offset* para mover el polígono a otra coordenadas diferentes del origen. Estas coordenadas se almacenan consecutivamente como una matriz 2D plana, utilizando la función anteriormente descrita *create2DArrayFlat*. Calcula los puntos a partir de las ecuaciones de las coordenadas polares 6.6, siendo t un ángulo interior que se calcula con la ecuación 6.7. Divide el ángulo completo entre el número de vértices multiplicado por el índice del vértice y le suma un ángulo constante. De esta forma, se calcula cada ángulo perteneciente a cada vértice y sus coordenadas asociadas.

$$x = r * cos(t), y = r * sin(t) \tag{6.1}$$

Figura 6.6: Ecuación de las coordenadas polares en dos dimensiones

$$t = \pi * i/n + \pi/2 \tag{6.2}$$

Figura 6.7: Ecuación del ángulo interno en un polígono

```
1 int createNGonVertices(size_t n, GLfloat xOffset, GLfloat yOffset,
    GLfloat r, GLfloat** verticesBuffer);
```

Extracto de código 6.4: Prototipos de la biblioteca de polígonos

El código 6.4 muestra el prototipo de la biblioteca en código *C*.

6.5.2. API pública

El usuario debería utilizar la función *createNGonVertices* para crear un polígono regular de n vértices, aportando un *buffer* donde almacenarlos y unas coordenadas bidimensionales que servirán de desplazamiento desde el centro de la pantalla.

Una vez calculados los vértices y sus coordenadas queda calcular la distribución de los triángulos dentro de este polígono. Pueden ser representados como un *triangle fan* y exactamente $n - 2$ triángulos; en OpenGL se asigna $GL_TRIANGLE_FAN$ en la función que dibuja el polígono. El primer vértice actúa de nodo principal y cada pareja de vértices posteriores se conecta al primero.

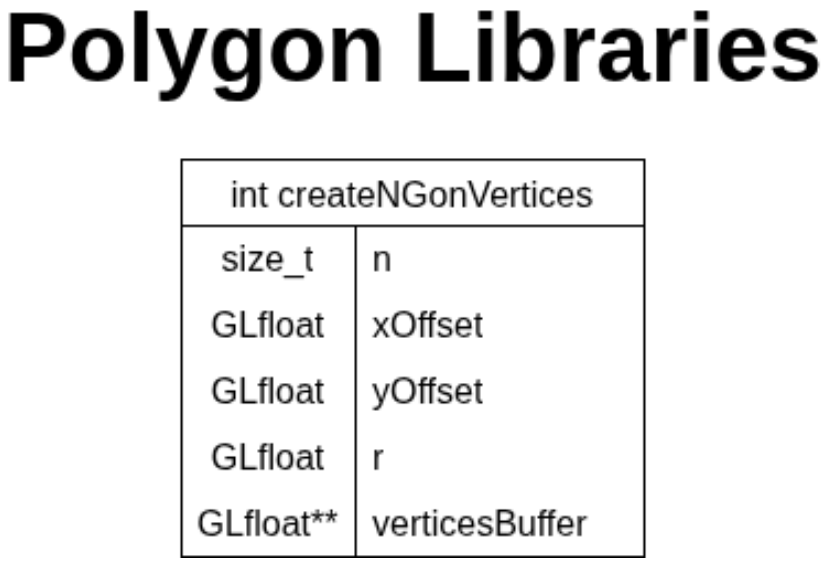


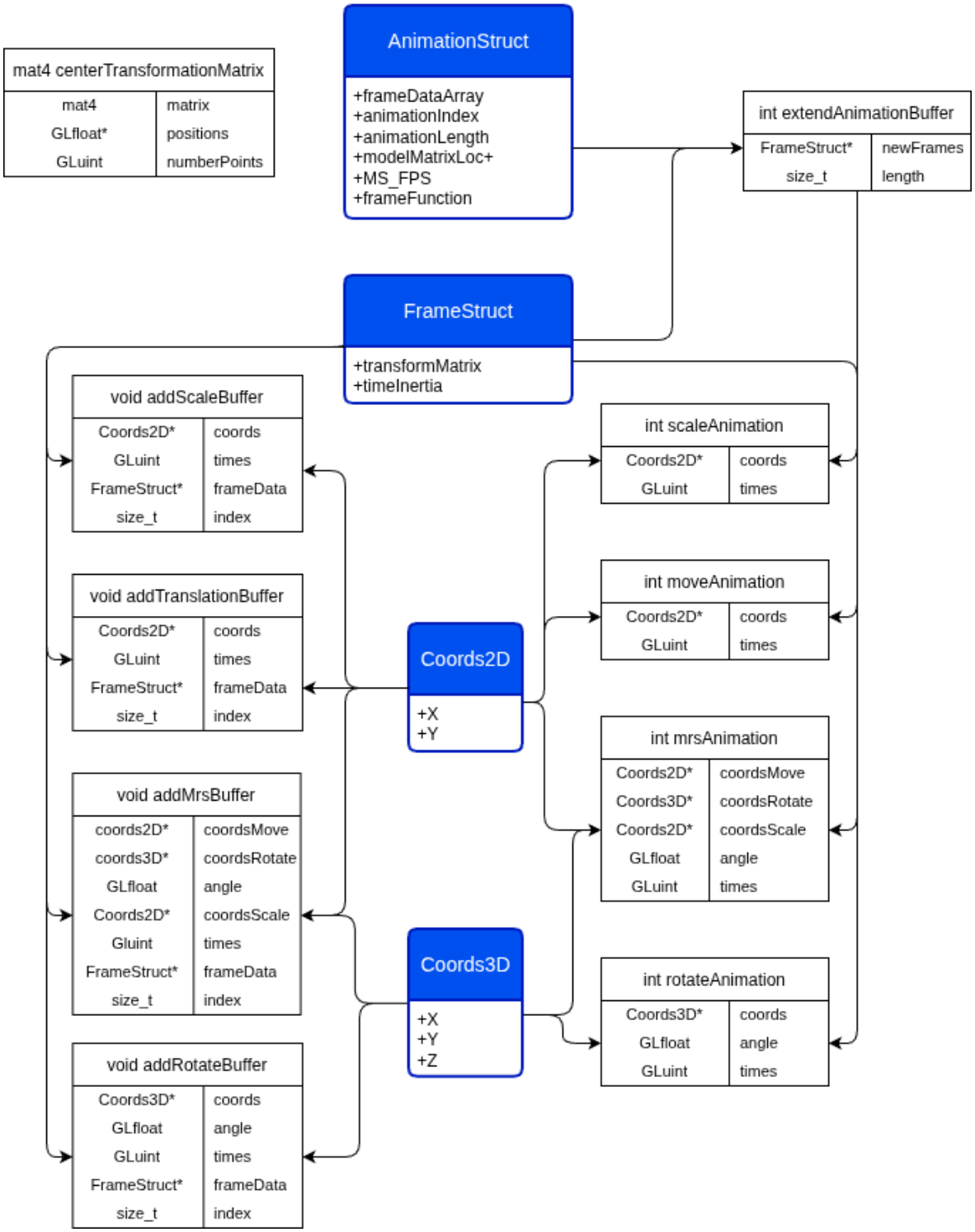
Figura 6.8: Diagrama de la biblioteca de polígonos.

6.6. Animaciones

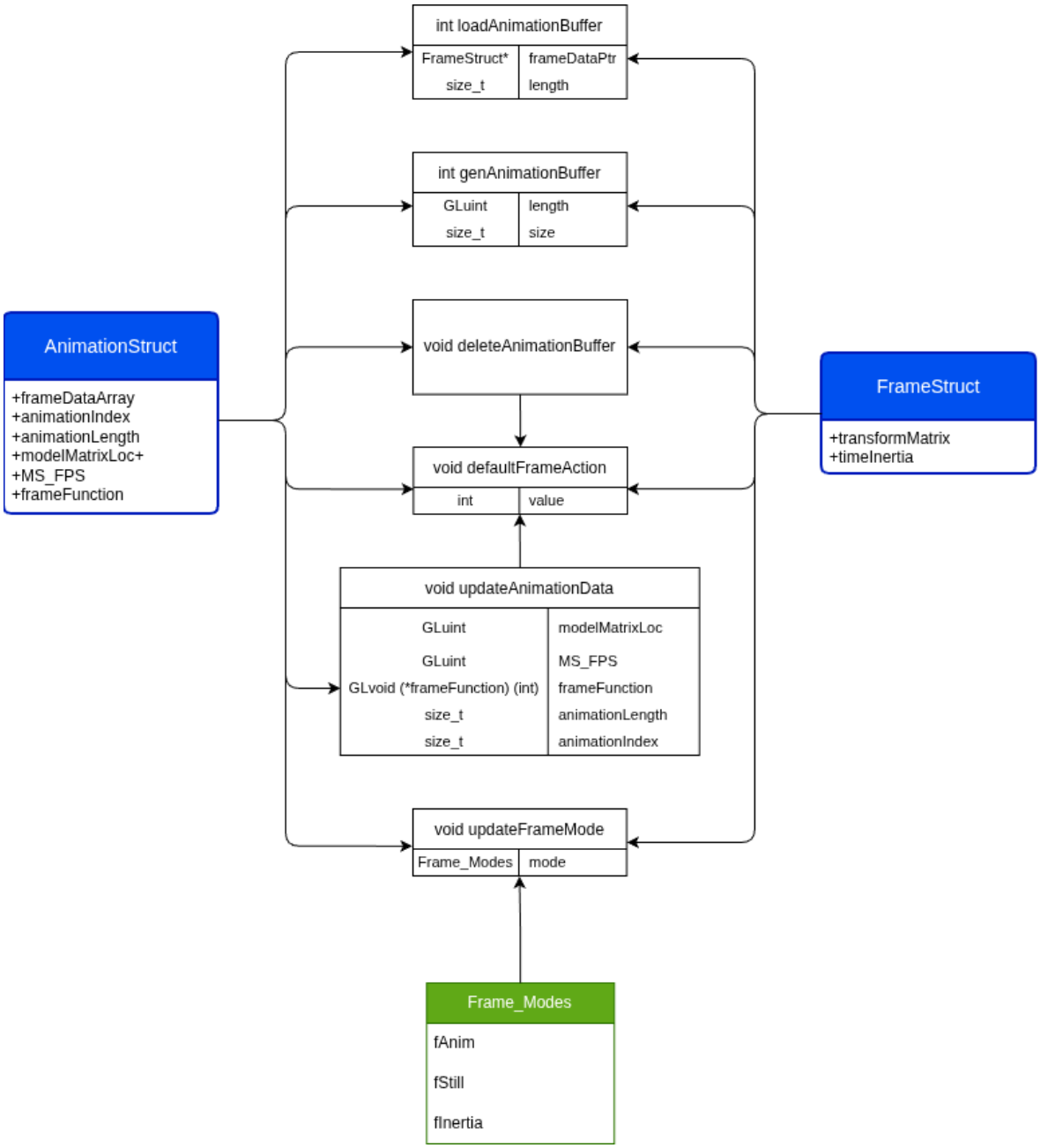
En esta sección se trata la biblioteca de **animaciones** 6.6 su motivación, funciones, estructuras de datos y la *API* pública que ofrece.

Se ha creado esta biblioteca con el propósito de facilitar la animación de geometría, permitiendo mover, escalar y rotar, ya sea cargando de una vez una serie de transformaciones o una a una. Gestiona los fotogramas y la acción en cada uno de ellos, aunque con un alto grado de personalización.

Animation libraries



(a) Diagrama de la biblioteca de animaciones; parte 1.



(a) Diagrama de la biblioteca de animaciones; parte 2.

Figura 6.10: Diagrama de la biblioteca de animaciones

6.6.1. Decisiones de diseño

El código 6.5 6.6 muestra el prototipo de la biblioteca en código *C*.

La primera parte del diagrama 6.9a muestra las funciones usadas para transformar la posición de la geometría:

- La estructura *AnimationStruct* describe la animación: fotogramas, número de repeticiones (inercia), *FPS*, etc. Para representar coordenadas, se han creado dos estructuras que describen coordenadas bidimensionales y tridimensionales.
- La estructura *FrameStruct* describe las características de un fotograma: la matriz de transformación asociada y el tiempo de inercia, que es el número de veces que se repetirá dicha transformación.
- Las estructuras *Coords2D* y *Coords3D* almacenan coordenadas bidimensionales y tridimensionales, respectivamente, en formato decimal
- Las funciones a la izquierda modifican un *buffer* local, usado más tarde para cargarlo en la animación:
    1. La función *addScaleBuffer* agrega a un *buffer* una matriz de transformación de escala.
    2. La función *addTranslationBuffer* agrega a un *buffer* una matriz de transformación de desplazamiento.
    3. La función *addMrsBuffer* agrega a un *buffer* una matriz de transformación de escala, desplazamiento y rotación, combinados.
    4. La función *addRotateBuffer* agrega a un *buffer* una matriz de transformación de rotación.
- Las funciones de la derecha modifican directamente la animación sin intermediario, lo que es más sencillo aunque costoso. También, debe estar configurada correctamente la animación antes de utilizarlas.
    1. La función *scaleAnimation* agrega al *buffer* de animación directamente una matriz de transformación de escala.
    2. La función *moveAnimation* agrega al *buffer* de animación directamente una matriz de transformación de desplazamiento.
    3. La función *mrsAnimation* agrega al *buffer* de animación directamente una matriz de transformación de escala, desplazamiento y rotación, combinados.
    4. La función *rotateAnimation* agrega al *buffer* de animación directamente una matriz de transformación de rotación.

La segunda parte del diagrama 6.10a muestra las funciones relacionadas a la configuración y las acciones de cada fotograma; concretamente, trata la configuración previa a la animación así como la limpieza posterior.

- El enumerado, llamado *Frame_Modes*, contiene 3 estados: quieto, animación e inercia. Cuando no está en movimiento, está **quieto**; cuando está en movimiento, está en **animación**; cuando sigue repitiéndose el movimiento, está en **inercia**, hasta que para y vuelve a **quieto**.
- La función *updateAnimationData* inicializa las propiedades de la animación almacenadas en una estructura *AnimationStruct* global.
- El trío de funciones *genAnimationBuffer*, *loadAnimationBuffer* y *deleteAnimationBuffer* trata el ciclo de vida de la animación: crea espacio inicial para la animación, deposita fotogramas y finalmente las destruye, respectivamente.
- La función *defaultFrameAction* indica lo que ocurre en cada fotograma. Por defecto, se selecciona una función ya existente que aplica las matrices de transformación cada fotograma tantas veces como se indique en el tiempo de inercia y avanza hacia el siguiente.
- Finalmente, la función *updateFrameMode* actualiza el estado de la animación y prepara un temporizador que activa el siguiente fotograma.

```
typedef struct FrameStruct
{
    glm::mat4 transformMatrix;
    size_t timeInertia;
} FrameStruct;

typedef struct
{
    FrameStruct* frameDataArray;
    size_t animationIndex;
    size_t animationLength;
    GLint modelMatrixLoc;
    GLuint MS_FPS;
    void (*frameFunction) (int);
} AnimationStruct;

typedef struct
{
    GLfloat x; GLfloat y;
} Coords2D;

typedef struct
{
    GLfloat x; GLfloat y; GLfloat z;
} Coords3D;

int moveAnimation(Coords2D* coords, GLuint times);
int scaleAnimation(Coords2D* coords, GLuint times);
int mrsAnimation(Coords2D* coordsMove, Coords3D* coordsRotate, Coords2D*
    coordsScale, GLfloat angle, GLuint times);
int rotateAnimation(Coords3D* coords, GLfloat angle, GLuint times);

void addRotateBuffer(Coords3D* coords, GLfloat angle, GLuint times,
    FrameStruct* frameData, size_t index);
void addScaleBuffer(Coords2D* coords, GLuint times, FrameStruct*
    frameData, size_t index);
void addTranslationBuffer(Coords2D* coords, GLuint times, FrameStruct*
    frameData, size_t index);
void addMrsBuffer(Coords2D* coordsMove,
        Coords3D* coordsRotate, GLfloat angle,
        Coords2D* coordsScale,
        GLuint times, FrameStruct* frameData, size_t index);

glm::mat4 centerTransformMatrix2D(glm::mat4 matrix, GLfloat *positions,
    GLuint numberPoints);
```

Extracto de código 6.5: Prototipos de la bibliteca de animación; parte 1

```
1  int loadAnimationBuffer(FrameStruct* frameDataPtr, size_t length);
2
3  int genAnimationBuffer(GLuint length, size_t size);
4
5  void deleteAnimationBuffer();
6
7  void defaultFrameAction(int value);
8
9  void updateAnimationData(GLuint modelMatrixLoc, GLuint MS_FPS,
10         GLvoid (*frameFunction) (int), size_t animationLength, size_t
       animationIndex);
11
12 void updateFrameMode(Frame_Modes mode);
13
14 typedef enum { fAnim, fStill, fInertia } Frame_Modes;
```

Extracto de código 6.6: Prototipos de la biblioteca de animación; parte 2

6.6.2. API pública

El usuario, según la manera en la que desee cargar los fotogramas de la animación, deberá utilizar las funciones que operan directamente sobre el *buffer* de animación o las que tratan primero con un *buffer* local para luego cargarlas en la animación.

En la primera opción, se asigna primero espacio en memoria al *buffer* de animación y se inicializa *AnimationStruct* mediante la función *updateAnimationData*. Luego, se crean listas de coordenadas *Coords2D* y *Coords3D* para ser usadas en las transformaciones y listas de fotogramas *FrameStruct*. Después, se programa la animación con las funciones *add*Buffer*, cargándolas en memoria mediante *loadAnimationBuffer*. Opcionalmente, puede extenderse la animación repitiendo el proceso o usando las funciones *rotate—scale—moveAnimation*.

Si no se desea cargar la animación de la primera forma, sólo se modificaría la llamada a *genAnimationBuffer*, indicando que el tamaño sería **cero**, y no se usarían las funciones *add*Buffer* y *loadAnimationBuffer*.

7. Implementación

En este capítulo se describen las tecnologías empleadas en el proyecto, detalles de implementación, problemas en el desarrollo y sus soluciones, y, en general, una vista a un nivel más bajo de cada parte del diseño.

Cabe recordar que la implementación está alojada en un repositorio de *Gitlab* [11].

7.1. Herramientas

Se han utilizado varias tecnologías. Se distinguirá entre tecnologías a nivel de sistema operativo, como pueden ser *CMake* o *git*, y tecnologías correspondientes al código directamente, como *GLUT* y *GLEW*.

Tecnologías usadas a nivel de *SO*:

- *CMake*, para gestionar la compilación del proyecto
- *git*, para el control de versiones del proyecto.
- *Debian*, distribución *GNU/Linux* sobre la que se ha desarrollado el proyecto.

Tecnologías usadas a nivel de implementación y código:

- *C* y *C++*, como lenguaje principal de programación.
- *OpenGL*, como *API* de gráficos.
- *GLM*, como biblioteca externa que facilita la implementación relacionada con matrices.
- *GLUT*, como biblioteca externa que facilita la gestión de eventos específicos al sistema operativo.
- *GLEW*, como biblioteca externa que facilita la gestión de las funciones de *OpenGL*.

7.2. Desarrollo de bibliotecas

En esta sección se detallarán las bibliotecas previamente descritas en el apartado de diseño, indicando los tipos de las variables, los valores recibidos y devueltos, comportamiento y uso intencionado; todo el desarrollo se describirá según el lenguaje de programación *C*.

7.2.1. Shaders

Según el diagrama de *shaders* 6.3, se crean dos funciones auxiliares que compilan y enlazan los *shaders*. La función principal es *loadShaders*, que compila los *shaders* recibidos y los une en un programa, haciendo uso de las dos funciones anteriores.

- La estructura *ShaderInfo* consta de dos datos constantes: el tipo del *shader*, siendo un enumerado de *OpenGL GLenum*, y la ruta hacia el fichero que contiene el *shader*, siendo una cadena de caracteres *char**.
- La función *compileShader* toma dos argumentos: Los contenidos del *shader* en una cadena de caracteres *GLchar** y un tipo de *shader GLenum*. Se le indica a *OpenGL* el tipo de *shader* a compilar y sus contenidos, devolviendo así un número entero identificador *GLuint* una vez compilado.
- La función *linkShader* toma una lista de *shaders* compilados *GLuint** y un número entero sin signo *GLuint* que representa la cantidad de *shaders* recibidos. Luego, se crea un programa y se enlazan a él todos los *shaders* compilados. Finalmente, se devuelve el identificador *GLuint* de este programa compilado.
- La función *loadShaders* toma una lista de *ShaderInfo ShaderInfo** y un número entero sin signo *GLuint* que representa la cantidad de *shaders* a procesar. Para cada *shader*, lee su contenido del fichero en la ruta correspondiente y lo compila según el tipo utilizando la función *compileShader*; los dos datos proporcionados por *ShaderInfo*. Una vez compilados, se enlazan en un sólo programa utilizando la función *linkShader*. Finalmente, se borran los *shaders* que no volverán a ser utilizados y se devuelve el identificador del programa final como número entero sin signo.

7.2.2. Errores

Según el diagrama 6.4, la función principal de esta biblioteca es *checkSuccess*, que recopila todos los errores y sus determinadas acciones en una instrucción *switch*. Recibe la dirección a un valor y un código de error. Mediante estos dos argumentos se especifica el error y se comprueba una condición de validez distinta. Cuando es erróneo, devuelve *-1*, en otro caso, *0*.

- Los códigos de error se reúnen en un *enum* llamado *Error_Types*.
- La función *checkSuccess* toma una dirección hacia un valor en memoria y un tipo de error *Error_Types*. Según el tipo del error se deduce el tipo del valor: se obtiene y se convierte al tipo deseado. Una vez se tiene el valor convertido correctamente, se comprueba con la condición del error esperada; por ejemplo, para comprobar un error cargando la animación, se valida que el valor no sea negativo. Finalmente, si hubo error se devuelve 1; en otro caso, se devuelve 0.
- Las macros *check_error*, *check_error_null* y *check_error_zero* comprueban el valor obtenido de *checkSuccess* y devuelven un valor correspondiente al que indique la función mediante una instrucción *return*: negativo, nulo o cero; se

asume que el negativo es el valor por defecto para representar errores, por lo que no se indica en la primera función. Es particularmente útil para evitar la repetición de condicionales e instrucciones de retorno, sólo debiendo llamar a estas *macros*.

- Las funciones *cast_ptr_int* y *cast_ptr_long* toman una dirección hacia un valor en memoria y devuelve ese mismo valor convertido en el formato correcto: *int* y *long*.

7.2.3. Misceláneo

Esta biblioteca 6.5 recoge funciones de propósito general de *C* sin usar *OpenGL* ni otras bibliotecas externas.

- La función *readFile* es útil para obtener el código de los *shader*; acepta una ruta como *array* de caracteres hacia un fichero y una dirección de memoria hacia un buffer (otro puntero a memoria, es decir, un doble puntero a memoria). La función abre el fichero y lo recorre para obtener su tamaño, asigna memoria al *buffer* y posteriormente vuelva el contenido del fichero en él, asegurándose de que el último byte es un carácter nulo, y se cierra el fichero.
- Las dos funciones restantes, *create2DArray* y *create2DArrayFlat*, crean un *array* de dos dimensiones en dos formatos distintos: *array* de *arrays* (bidimensional) o un *array* plano (unidimensional).

7.2.4. Polígonos

La única función actual de la biblioteca 6.8 crea un polígono regular, en forma de un *array* de vértices plano. Utilizando la ecuación paramétrica de un círculo, construye las coordenadas de cada vértice calculando los ángulos correspondientes. Esta función es especialmente útil como base de pruebas, o *benchmark*, porque es sencillo estimar los triángulos que contiene un polígono regular de n vértices.

El programa construye un polígono regular de n vértices en el centro de la pantalla, como está detallado en la sección 6.5. Cualquier polígono regular es convexo [48], por lo cual pueden ser representados como un *triangle fan* [15, Capítulo 3] y exactamente $n - 2$ triángulos; en OpenGL se cambia *GL_TRIANGLES* a *GL_TRIANGLE_FAN* en la función que dibuja el polígono. El primer vértice actúa de nodo principal y cada pareja de vértices posteriores se conecta al primero.

7.2.5. Animaciones

Esta biblioteca contiene lo necesario para crear animaciones con *OpenGL*. La primera parte del diagrama 6.9a muestra las funciones usadas por el usuario para transformar la posición de la geometría. Las funciones a la izquierda modifican un

buffer local, usado más tarde para cargarlo en la animación. Las funciones de la derecha modifican directamente la animación sin intermediario, lo que es más sencillo aunque costoso.

- La estructura *FrameStruct* contiene los datos de un **fotograma**.
    1. *transformMatrix*. La matriz de transformación 4x4 de números de punto flotante que se aplica a las coordenadas de los vértices en cada fotograma
    2. *timeInertia*. El tiempo de inercia, un número entero que indica las veces que se repetirá la transformación en los siguientes fotogramas.
- La estructura *AnimationStruct* contiene los datos de la animación completa, especialmente una lista de *FrameStruct*.
    1. *frameDataArray*. Una lista de fotogramas con la estructura *FrameStruct*.
    2. *animationIndex*. Un número entero sin signo *size_t* que representa la posición actual de la animación.
    3. *animationLength*. Un número entero sin signo *size_t* que representa el tamaño de la animación.
    4. *modelMatrixLoc*. Un número entero *GLint* que representa la localización de la variable de tipo *uniform* que almacena la matriz de transformación en el *shader*.
    5. *MS_FPS*. Un número entero sin signo que representa los fotogramas por segundo transformados en milisegundos.
    6. *frameFunction*. Un puntero hacia una función que acepta un argumento entero sin devolver un valor. Es el mecanismo principal del fotograma que se ejecuta por cada uno y se encarga de realizar la transición entre los estados de la animación, gestionar las matrices de transformación y la comunicación entre el programa y el *shader*.
- Para representar coordenadas, se han creado dos estructuras *Coords2D* y *Coords3D* que almacenan coordenadas de punto flotante bidimensionales *(X,Y)* y tridimensionales *(X,Y,Z)*.
- La función *extendAnimationBuffer* extiende la animación principal con fotogramas nuevos. Acepta una lista de fotogramas de tipo *FrameStruct* llamada *newFrames* y un tamaño de tipo *size_t* llamado *length*. Se reasigna la memoria asociada al *buffer* de animación principal según el nuevo tamaño, actualizando así el tamaño de la animación, y se copian los nuevos fotogramas. Es principalmente usada en las funciones de la derecha: *scaleAnimation*, *moveAnimation*, *mrsAnimation* y *rotateAnimation*.
- Las funciones *addScaleBuffer*, *addTranslationBuffer*, *addRotateBuffer* y *addMrsBuffer* añaden una matriz de transformación de escala, traslación, rotación o todas al mismo tiempo, respectivamente, a la lista de fotogramas en una posición *index* concreta para luego ser procesados en la animación.

    - Las funciones *addScaleBuffer* y *addTranslationBuffer* toman unas coordenadas bidimensionales de tipo *Coords2D*, un número entero sin signo de veces que se repite la transformación, una lista de fotogramas de tipo *FrameStruct* y un índice donde insertar la matriz en la lista de fotogramas.

- Finalmente, se añadió una función *centerTransformationMatrix*, que acepta una matriz 4x4 de números flotantes, una lista de *GLFloat* con los vértices del polígono y un número entero con la cantidad de vértices. Realiza cambios en una matriz de transformación para que rote respecto a sí mismo y no respecto al origen de coordenadas. Para ello, se hace la media de las coordenadas y se encuentra el centro del polígono, respecto al que se realizarán las transformaciones en lugar del origen de coordenadas. Puede usarse si el *shader* no incorpora esta función.

La segunda parte del diagrama 6.10a muestra las funciones relacionadas con *buffers* y las acciones de cada fotograma, es decir, la configuración de la animación.

- El *enum* llamado *Frame_Modes* contiene 3 estados: quieto (fStill), animación (fAnim) e inercia (fInertia), útiles para representar los estados en los que se encuentra el programa.
- La función *updateAnimationData* inicializa las propiedades de la animación almacenadas en una estructura *AnimationStruct* global. Recibe como argumentos todos los valores de la estructura y se los asigna directamente a la variable global de la biblioteca.
- El trío de funciones *genAnimationBuffer*, *loadAnimationBuffer* y *deleteAnimationBuffer* gestiona el ciclo de vida de la animación.
    - La función *genAnimationBuffer* asigna espacio inicial en memoria para la animación.
    - La función *loadAnimationBuffer* copia fotogramas en memoria.
    - La función *deleteAnimationBuffer* las destruye liberando la memoria asignada.
- La función *defaultFrameAction* es la acción por defecto que ocurre en cada fotograma.
    - En el estado *fAnim*, se obtienen de memoria los datos del fotograma y se guardan en una variable global, se realiza una operación de multiplicación de matrices entre la matriz de inercia acumulada durante toda la animación y la matriz de transformación del fotograma actual y luego se copia a la variable *uniform* del *shader*. Una vez copiado el resultado, se renderiza el nuevo fotograma y se aumenta el índice de animación en uno.
    - En el estado *fInertia*, se realiza la operación entre matrices como en el estado *fAnim*, aunque usando la matriz de la variable global del fotograma anterior. Se copia el resultado al *shader* y se renderiza. Al final, se reduce el tiempo de inercia *timeInertia* en uno.

- En el estado *fStill* se libera la memoria del *buffer* de animación y no se realiza ninguna operación.

En el diagrama 7.1 se representa la función por defecto que se aplica cada fotograma. Cada tabla tiene como título el estado que representa y cada entrada tiene el índice y la operación que se realiza. Las flechas indican el movimiento entre estados, desde el estado origen hacia el estado destino, con un número que indica el orden que se sigue para comprobar la condición de la transición que se encuentra junto a la flecha.

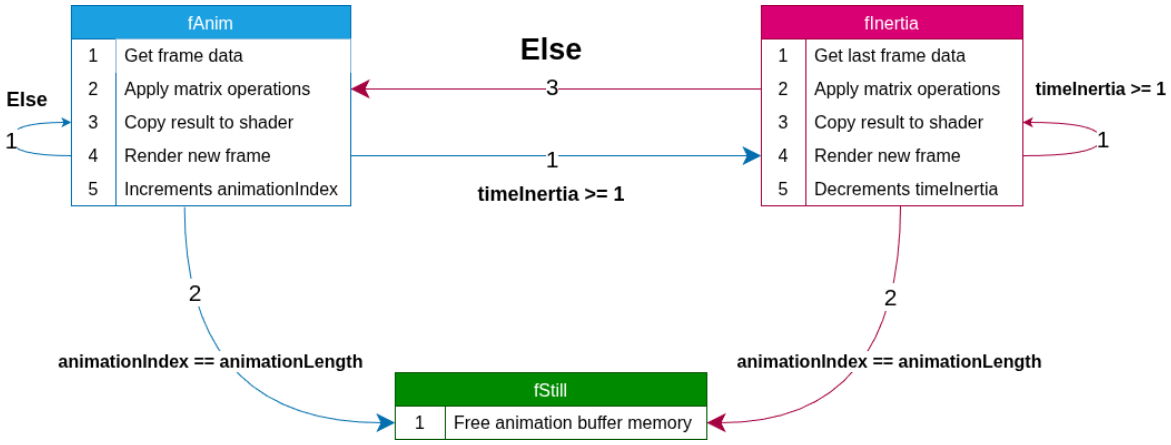


Figura 7.1: Diagrama de la función por defecto por fotograma.

- Finalmente, la función *updateFrameMode* actualiza el estado de la animación y prepara un temporizador que activa el siguiente fotograma.

7.2.6. Ejemplo de uso

Con el objetivo de ilustrar la cooperación entre las diferentes bibliotecas creadas, se diseñará un programa de ejemplo, a alto nivel, cuyo propósito sea crear una animación con un polígono regular. El objetivo es generar matrices de transformación que se enviarán al *shader* en forma de variable *uniform*.

El programa está representado a alto nivel en el diagrama 7.2. Los rectángulos coloreados son los que incluyen funcionalidades de las bibliotecas. Aunque no representado para simplificar el diagrama, se utiliza la comprobación de errores de la biblioteca cuando sea posible en todo el programa.

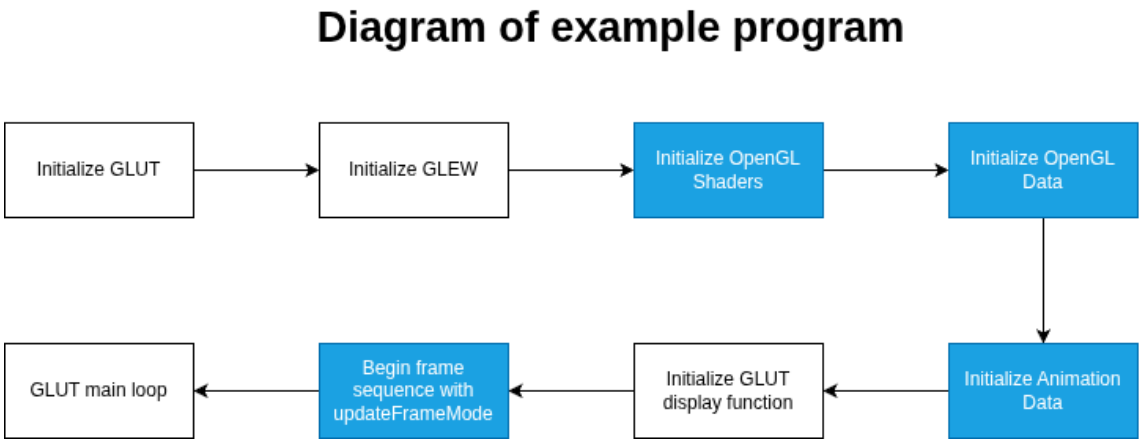


Figura 7.2: Diagrama del programa de ejemplo.

A continuación, se describe la rutina de inicialización de *shaders* y cómo se usa la biblioteca para procesarlos y crear un programa. Primero, se crea un *array* de tipo *ShaderInfo* de una longitud del número de *shaders* a procesar, con el tipo de *shader* y la ruta hacia el fichero con su contenido. Este *array* es recibido por la función *loadShaders* y éste genera el **programa** ya procesado. Se obtiene la localización de la variable *uniform* que contiene la matriz de transformación y se activa el programa.

Además del código del programa escrito en *C*, es necesario escribir los *shaders* correspondientes en la ruta */shaders* del proyecto; la extensión *.vert* y *.frag* son pertenecientes a los *shaders vertex* y *fragment*.

- El *vertex shader* es responsable de realizar la operación matricial que modifica la posición del vértice.
    1. Contiene un vector bidimensional llamado **vPosition** que contiene las coordenadas del vértice y una matriz 4x4 *uniform* de tipo *mat4* llamada **modelMatrix** que contiene la matriz de transformación.
    2. Se crea una matriz 4x4 llamada *centerMatrix* que multiplica a *modelMatrix* para que realice la transformación entorno a sí mismo.
    3. Finalmente, se multiplican las coordenadas del vértice a la matriz de transformación centrada y se obtiene la nueva posición del vértice *gl_Position*.
- El *fragment shader* es un *shader* simple que asigna el color blanco a *fragColor*, que es un vector de 4 dimensiones.

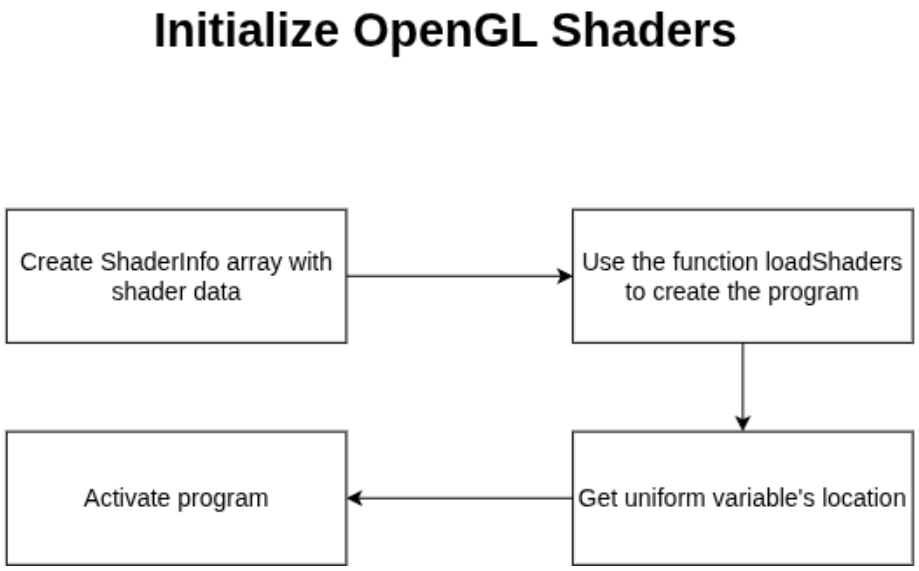


Figura 7.3: Diagrama de inicialización de shaders en el programa de ejemplo.

El diagrama 7.4 representa el proceso para crear los datos que usará *OpenGL* para dibujar la geometría.

1. Se crea un polígono regular mediante la función de la biblioteca de polígonos *createNGonVertices*, indicando el número de vértices deseados, sus coordenadas, dimensiones y el *buffer* donde se almacenarán los vértices del polígono.
2. Deberán crearse los *buffers* de *OpenGL*, los cuales en este caso son el *VAO* y el *VBO*, mediante el par de funciones (para cada objeto) *glGenVertexArrays* y *glBindVertexArray*; generan internamente el objeto y luego lo vincula.
3. Luego, se almacenan los vértices en el *VBO* mediante la función *glBufferData*, indicando el tipo de *buffer*, el tamaño en *bytes* a copiar, el origen y el tipo de uso que en este caso sería *GL_DYNAMIC_DRAW*.
4. Después, se especifican las posiciones de los atributos de los vértices con *glVertexAttribPointer* y se activan con *glEnableVertexAttribArray*.
5. También, se inicializa la variable *uniform* en el *shader* que se utiliza para la matriz de transformación con una matriz de identidad.
6. Finalmente, se libera el *buffer* de vértices creado inicialmente para alojar el polígono.

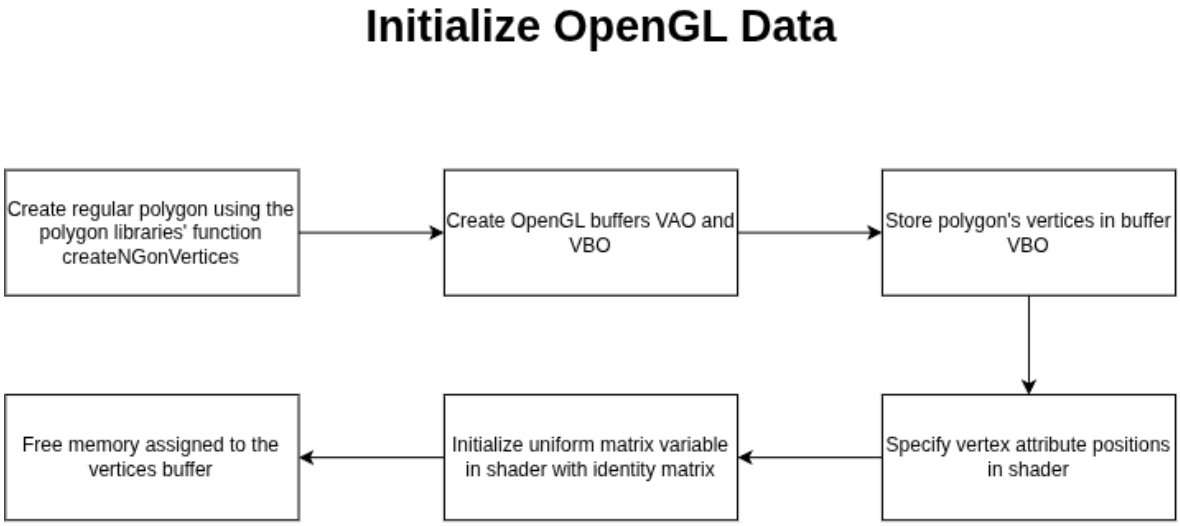


Figura 7.4: Diagrama de inicialización de datos para OpenGL.

Finalmente, se muestra la inicialización de los datos de animación. Primero, se asigna memoria para el *buffer* de animación con *genAnimationBuffer*, indicando el tamaño en *bytes* y el número de elementos. Luego, se actualizan los atributos de la estructura de datos *AnimationStruct* mediante la función *updateAnimationData*, que recibe como argumentos las propiedades y se las asigna. Se crean, también, dos *arrays* para coordenadas bidimensionales *Coords2D* y tridimensionales *Coords3D* para usar en las transformaciones.

Se crea un *buffer* de fotogramas, que se completará usando funciones del tipo *add*Buffer*, declaradas en la biblioteca de animación; estas funciones añaden fotogramas *FrameStruct* a un *buffer*, indicando el tipo de transformación, sus características y el índice en el que colocar el fotograma. Una vez lleno el *buffer* de fotogramas, se guardan en memoria con *loadAnimationBuffer*. Opcionalmente, puede extenderse la animación usando las funciones *rotate—scale—moveAnimation*, indicando las características de la transformación y el fotograma, que automáticamente colocan la transformación correspondiente en la secuencia de animación.

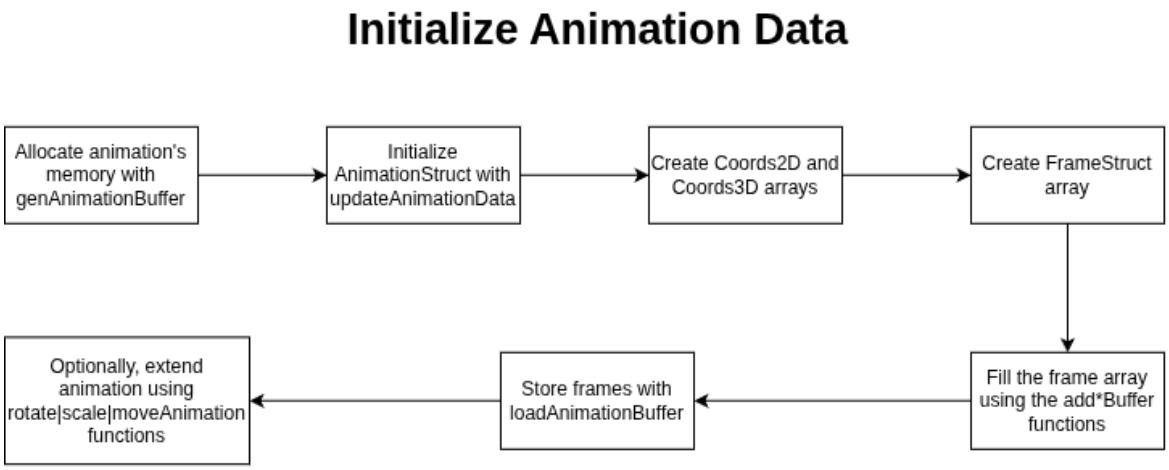
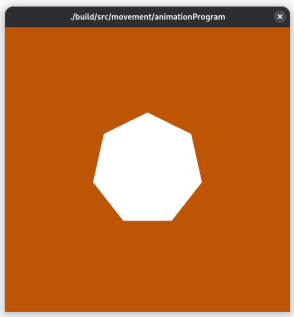
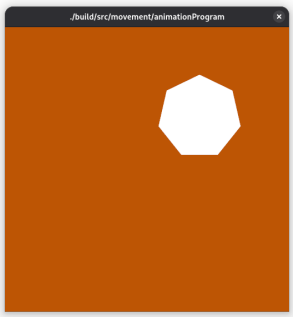


Figura 7.5: Diagrama de inicialización de los datos para la animación.

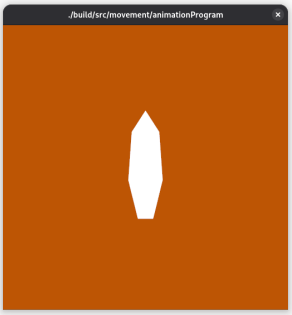
7.2.7. Demostración



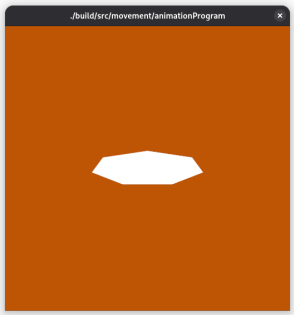
(a) Polígono parado.



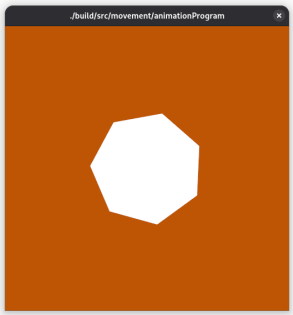
(b) Polígono en movimiento.



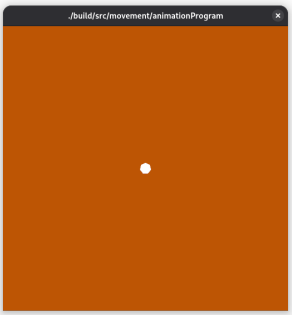
(c) Polígono rotando entorno al eje y.



(d) Polígono rotando entorno al eje x.



(e) Polígono rotando entorno al eje z.



(f) Polígono encogiendo.

7.3. Conclusiones

En esta sección se ha explorado con detalle las bibliotecas y las funciones y estructuras que las componen. También, se ha ofrecido un ejemplo de uso con el objetivo de que el lector pueda encajar las diferentes partes del proyecto y conceptualizar un resultado.

8. Pruebas

Las optimizaciones utilizadas durante las pruebas fueron las provocadas por la *flag O3* en el compilador *GCC* [49].

En este capítulo se describen las pruebas realizadas sobre los programas desarrollados en el proyecto, así como el análisis de los resultados desde diferentes perspectivas, con el fin de evaluar aspectos como los valores de las métricas obtenidas, la uniformidad de los resultados, el comportamiento general y el rendimiento. Para ello, las versiones empleadas en las pruebas incorporan código adicional específico para caracterizar el rendimiento, generando dos ficheros: *.log*, que recoge información legible para el usuario, y *.dat*, que contiene los datos necesarios para la representación gráfica. Para las pruebas de rendimiento de vértices en un polígono 8.1 y de animación 8.2, los ficheros *.dat* son procesados mediante *gnuplot* y genera las gráficas correspondientes.

Se han realizado pruebas sobre el uso de memoria 8.3 con *Valgrind* [50].

También, se ha realizado un análisis más profundo de *CPU* 8.4 con *perf* [51].

Las pruebas se han ejecutado en el siguiente entorno *hardware* y *software*:

- El sistema operativo es una distribución de *GNU/Linux*, concretamente, *Debian 13, Trixie* [13].
- La *CPU* es *Intel(R) Core(TM) i5-7400 (4) @ 3.50 GHz*.
- La *GPU* es *NVIDIA GeForce GTX 1050 [Discrete]*.
- La versión de *OpenGL* es: 4.6.0 NVIDIA 550.163.01.
- La versión del compilador *gcc* es gcc (Debian 14.2.0-19) 14.2.0.
- La versión de *make* es GNU Make 4.4.1.

8.1. Pruebas de rendimiento de creación de polígonos

La prueba diseñada para probar el rendimiento de la creación de polígonos con una alta cantidad de vértices ha sido realizar iteraciones en un bucle en el que se aumenta progresivamente su número, y se analiza el rendimiento del programa.

El funcionamiento del núcleo del programa, sin profundizar en configuración ni otros mecanismos ya explicados anteriormente y no relevantes para la prueba, es muy simple: en un bucle, con un número determinado de vértices inicial, crea un polígono regular de ese número de vértices que luego se renderiza. En la siguiente iteración, se realiza la misma operación con un vértice más.

8.1.1. Problemas encontrados

En la prueba de los vértices de un polígono el principal problema fue cómo visualizar la evolución del rendimiento al incrementar progresivamente los vértices de un polígono. Finalmente, se decidió desarrollar un único programa que cada fotograma aumente el número de vértices y renderice el polígono de nuevo.

Una vez realizada la prueba era necesario medir los tiempos y escribir los registros. Se crearon dos tipos de fichero para este propósito: uno que registra datos en crudo para las gráficas y otro más legible para el usuario. El mayor inconveniente era el dónde situar estas medidas, y entonces se decidió medir la creación del polígono y la renderización del mismo que realizan la funciones *initGLData* y *display*; este es el comportamiento de la *CPU* y la *GPU*, respectivamente.

8.1.2. Resultados

Se muestra una captura de pantalla tomada durante la prueba 8.1.

Utilizando los datos generados por la prueba se han creado varias gráficas que representan el rendimiento de la *CPU* y la *GPU* durante la ejecución. La primera gráfica representa el coste temporal que requiere crear los polígonos cuyos vértices aumentan progresivamente, desde **3** hasta **1000**, y la segunda y tercera gráfica representan la *CPU* y la *GPU* durante la renderización.

Observando la gráfica 8.2, se puede deducir a simple vista que es lineal. Con los resultados de diferentes medidas realizadas sobre estos datos en la tabla 8.1 puede resaltarse:

- Dado que $media > mediana > moda$, se deduce que los datos están sesgados positivamente y se acumulan hacia la izquierda de la gráfica.
- La desviación estándar y la varianza grandes indican que los datos difieren de la media bastante: que la varianza esté cerca de la mitad de la escala no es coincidencia, ya que la gráfica sigue una distribución lineal.
- El valor del *IQR* confirma la anterior deducción: los datos en el medio de la gráfica tienen una dispersión del 3,43, que es aproximado a la mitad de la escala.
- La correlación entre los datos de vértices y el tiempo de *CPU* es cercana a 1: siendo el crecimiento del número de vértices lineal, también lo sería el rendimiento.



Figura 8.1: Captura de pantalla de la prueba del polígono de 3 a 1000 vértices

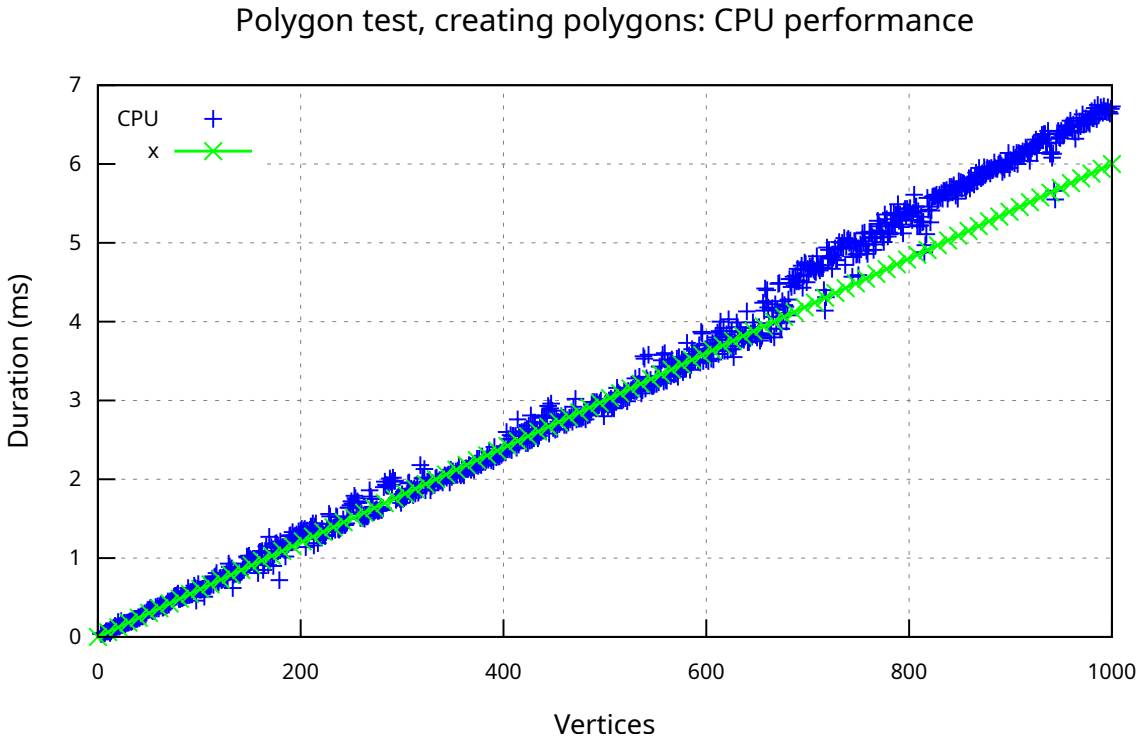


Figura 8.2: Gráfica que representa el rendimiento de la *CPU* en la prueba del polígono durante su creación.

Media:	3.203347
Mediana	2.975000
Moda	2.518307
IQR	3.437500
Varianza	3.822103
Desviación estándar	1.955020
Covarianza (v)	561.004122
Correlación (v)	0.995537
Q1	1.5425
Q2	2.9750
Q2	4.9800

Tabla 8.1: Resultados de la creación de vértices en la prueba del polígono

El par de gráficas 8.3 representan el rendimiento durante la renderización por parte de la *GPU* y la *CPU*. Las dos gráficas muestran un comportamiento logarítmico, aunque la primera se ajusta más a una función como $log(x)$ y la segunda como $log(log(x))$. Pueden observarse en los resultados de la *GPU* 8.2:

- La desviación estándar es muy pequeña respecto a los datos, lo que quiere decir que el *jittering* es mínimo y los los datos no difieren mucho respecto a la media.
- Dado que $media > mediana > moda$, se deduce que los datos están sesgados negativamente y se acumulan hacia la izquierda de la gráfica.
- La covarianza mayor a 0 entre la *CPU* y los vértices indica una dependencia entre los datos positiva.
- La correlación entre la *CPU* y los vértices está entre 0 y 1, aunque acercándose a este último. Esto indica una relación positiva y lineal entre estas dos variables.
- Los cálculos de covarianza y correlación cercanos a 0 entre la *CPU* y la *GPU* indican que los dos datos son independientes y no hay relación entre ellas.

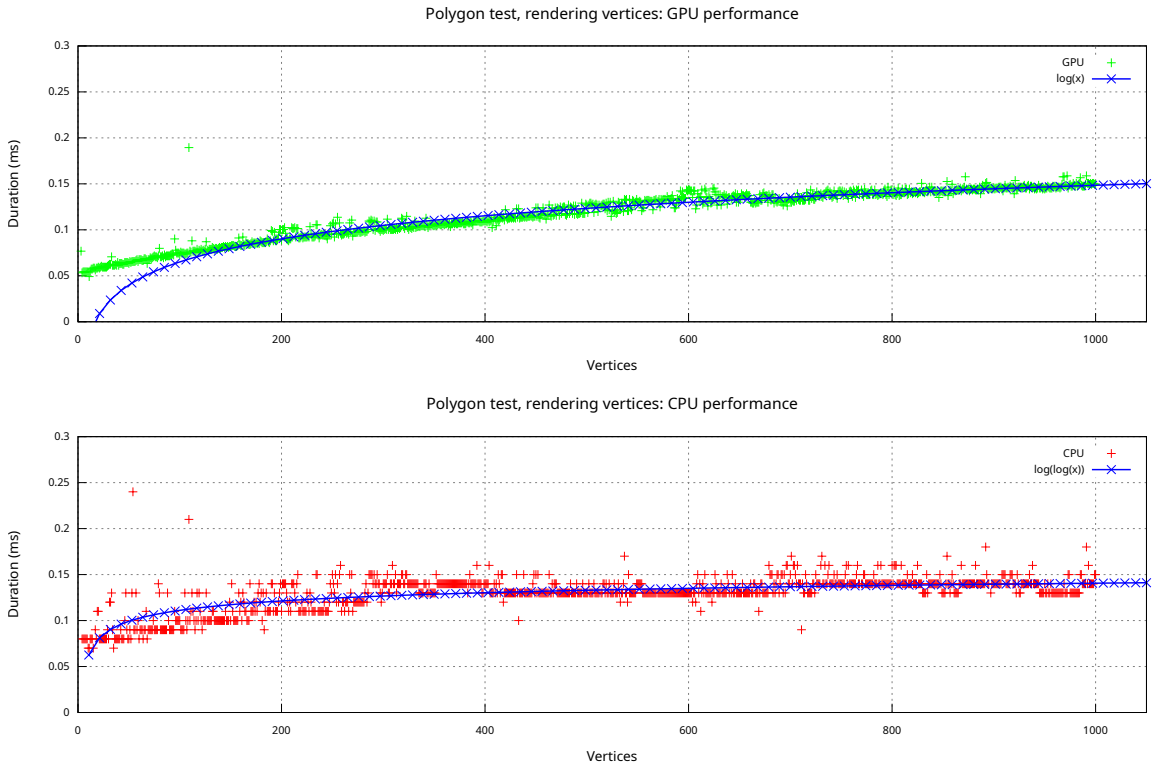


Figura 8.3: Gráficas que representan el rendimiento de la *GPU* y la *CPU* en la prueba del polígono durante la renderización.

Media:	0.118084
Mediana	0.123392
Moda	0.134008
IQR	0.040960
Varianza	0.001664
Desviación estándar	0.040793
Covarianza (v)	7.548455
Correlación (v)	0.641963
Covarianza (GPU)	0.001354
Correlación (GPU)	0.759181
Q1	0.098304
Q2	0.123392
Q3	0.139264

Tabla 8.2: Resultados de la renderización en la prueba del polígono: **GPU**

Pueden observarse en los resultados de la *CPU* 8.3:

- La varianza y desviación típica son mayores que en las de la *GPU* y son relativamente grandes respecto a la escala de los datos. Esto indica una alta dispersión en los resultados respecto a la media.
- El *IQR* es cercano a 0, lo que indica una similitud entre cuartiles y, a su vez, mínima diferencia en los resultados.
- La covarianza entre la *CPU* y los vértices indica una dependencia negativa entre estas dos variables: cuando aumentan los vértices, disminuye la duración de la *CPU*.
- La correlación entre la *CPU* y los vértices es cercana a 0, lo que indica que existe independencia lineal entre estas dos variables.
- Los cálculos de covarianza y correlación cercanos a 0 entre la *CPU* y la *GPU* indican que los dos datos son independientes y no hay relación entre ellas.

Media	0.132385
Mediana	0.130000
Moda	0.125230
IQR	0.010000
Varianza	0.001912
Desviación estándar	0.043723
Covarianza (v)	3.647974
Correlación (v)	0.289460
Covarianza (GPU)	0.001354
Correlación (GPU)	0.759181
Q1	0.13
Q2	0.13
Q3	0.14

Tabla 8.3: Resultados de la renderización en la prueba del polígono: **CPU**

8.2. Pruebas de rendimiento de animaciones

Esta prueba se ha realizado con 2^{10} polígonos de **32** lados, o también llamados **icosidodecágonos**, desplegados por la pantalla. Para conseguir un escenario sencillo y previsible sobre la prueba, se decidió agrupar los polígonos en una matriz cuadrada y siempre la cantidad siendo 2^x, donde x es un número par con el objetivo de construir una matriz cuadrada de tamaño $2^{(x/2)}$ por $2^{(x/2)}$. Cada polígono obtiene sus coordenadas en función del número de polígonos a crear y de las dimensiones de la ventana. El diámetro de cada polígono se calcularía entonces como $\frac{1}{P}$, siendo P el número de polígonos por fila o columna, para que cubran toda la ventana sin colisionar entre ellos, calculado como $P = 2^{\frac{M}{2}}$; Luego, considerando que en *OpenGL* el origen de coordenadas está en el centro:

$(x, y) = (-\frac{1}{2} + r + d \times i, \frac{1}{2} - (r + d \times j))$. El diámetro de cada polígono d se calcula de manera inversamente proporcional a P: $d = \frac{1}{p}$. De esta forma, se coloca cada polígono en cada casilla de la matriz. Con estas coordenadas calculadas, puede llamarse a la función *createNGonVertices* para crear el polígono regular.

Algorithm 1 Algoritmo de creación de polígonos en matriz cuadrada

Require: M is **pair** and $N > 0$ ▷ N is the number of vertices of each polygon
  $P \leftarrow 2^{\frac{M}{2}}$ ▷ P is the number of polygons per row or column
  $d \leftarrow \frac{1}{P}$ ▷ d is the diameter of each polygon
  $r \leftarrow \frac{d}{2}$
  while $j < P$ **do**
    while $i < P$ **do**
      $x \leftarrow -\frac{1}{2} + r + d \times i$
      $y \leftarrow \frac{1}{2} - (r + d \times j)$
  `createNGonVertices(`N`, `x`, `y`, `r`, &(vertices[`$P \times j + i$`]))`
    end while
  end while

8.2.1. Problemas encontrados

En la prueba de animación, el principal problema residía en la naturaleza de la prueba a realizar; la cual consistía en reproducir una animación donde la mayor cantidad de polígonos actuaran a la vez en una misma escena, moviéndose al unísono, para comprobar cómo afectaría eso al rendimiento. Para poder representar todos estos polígonos a la vez y que fueran visibles, era necesario agruparlos correctamente y que no ocultasen unos a otros. Entonces, se calcularon las coordenadas de cada polígono como si de una matriz se tratase y cada figura un elemento de ella, con unos márgenes entre ellos para separarlos mínimamente.

En esta prueba la posición en la que colocar las mediciones de tiempo era menos intuitiva. Ya que el objetivo principal era medir el rendimiento de la *GPU*, se mide el tiempo entre fotogramas; asimismo, el rendimiento de la *CPU* se mide en el mismo intervalo para poder comparar. También, fue útil medir el tiempo que tardaba la *CPU* en crear todos los polígonos usados en la animación.

8.2.2. Resultados

Se muestra una captura de pantalla de la prueba de animación en funcionamiento 8.4.

Los resultados de la *CPU* 8.5 durante la creación de polígonos indican simetría y mínima dispersión:

- Dado que $media < mediana < moda$, se puede deducir que la distribución de los datos está sesgada negativamente y se acumulan ligeramente hacia la izquierda.

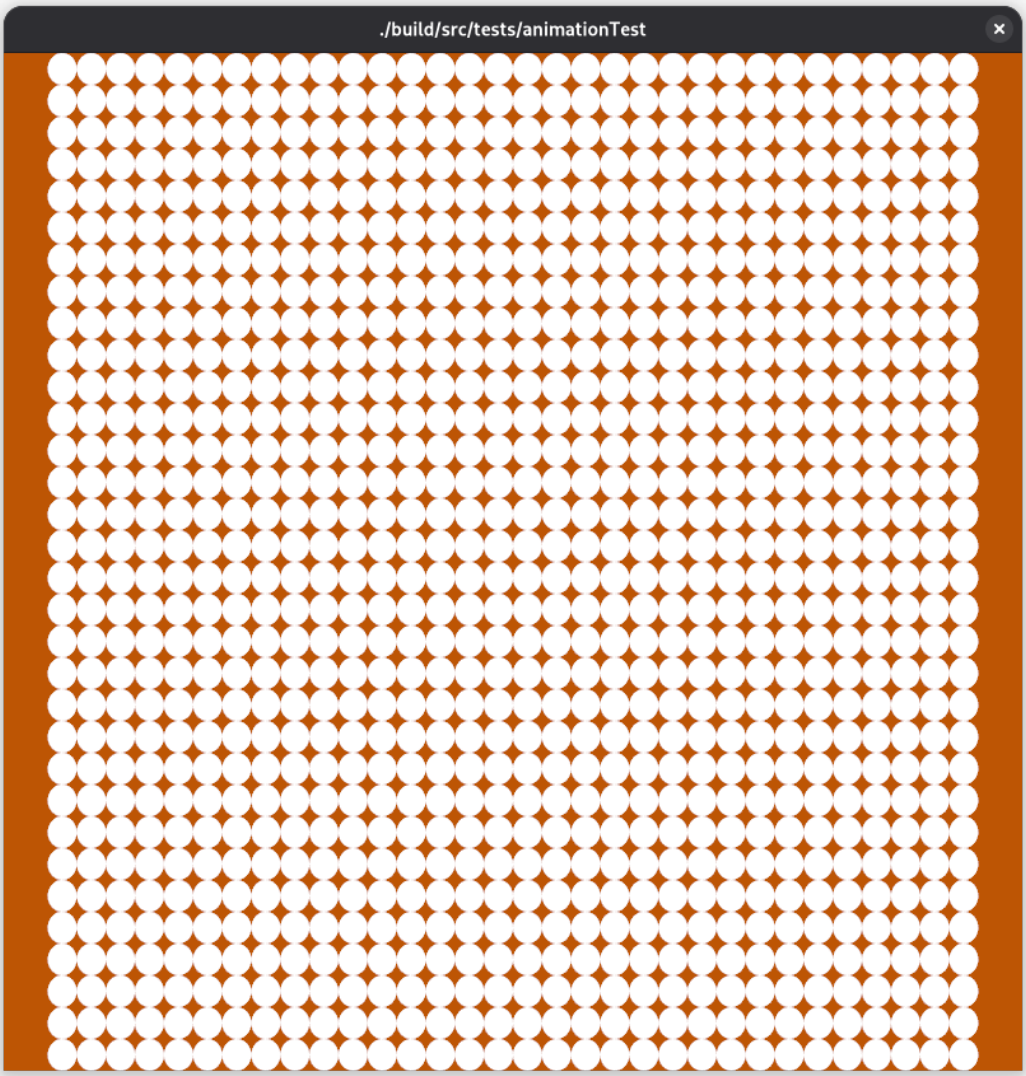


Figura 8.4: Captura de pantalla de la prueba de la animación de 1024 polígonos en una matriz cuadrada.

- El *IQR* calculado a partir de los cuartiles indica que la dispersión de los valores en la mitad de la gráfica tienen una dispersión de 0,03, lo que es despreciable en esta escala.
- Los valores de la variación y la desviación estándar son pequeños, lo que indica poca dispersión respecto a la media.

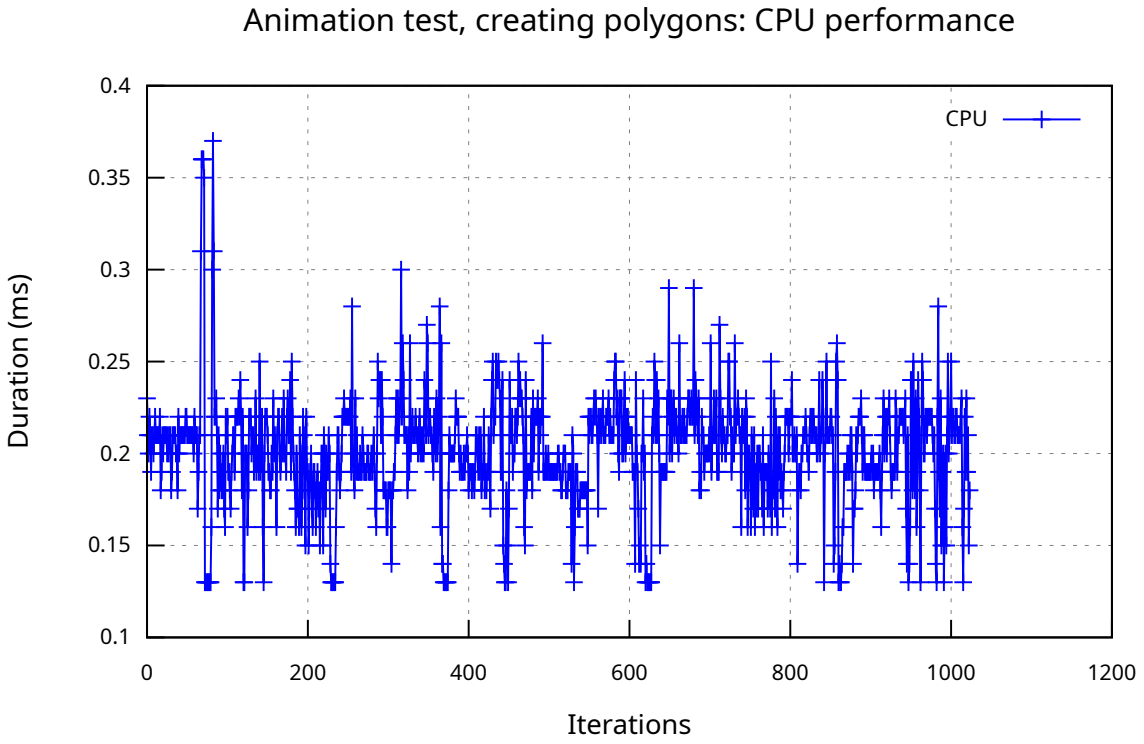


Figura 8.5: Gráfica que representa el rendimiento de la *CPU* en la prueba de animación durante la creación de los polígonos.

Media:	0.202725
Mediana	0.210000
Moda	0.224551
IQR	0.030000
Varianza	0.000965
Desviación estándar	0.031060
Q1	0.19
Q2	0.21
Q3	0.22

Tabla 8.4: Resultados de la creación de polígonos en la prueba de animación (**CPU**)

Pueden observarse en los resultados de la *CPU*:

- Dado que $media > mediana > moda$, se deduce que los datos están sesgados positivamente y se acumulan hacia la izquierda de la gráfica.

- Los valores de la varianza y la desviación típica son muy pequeños, lo que indica que los datos difieren muy ligeramente de la media.
- El *IQR* con valor 0,047 indica que la dispersión de los valores en la mitad de la gráfica es diminuta y su distribución es uniforme.
- La covarianza entre la *CPU* y los vértices indica una dependencia negativa entre estas dos variables: cuando aumentan los vértices, disminuye la duración de la *CPU*.
- La correlación entre la *CPU* y los vértices es cercana a 0, lo que indica que existe independencia lineal entre estas dos variables.
- Los cálculos de covarianza y correlación cercanos a 0 entre la *CPU* y la *GPU* indican que los dos datos son independientes y no hay relación entre ellas.

La gráfica de la *GPU* 8.6 es interesante, ya que pueden distinguirse los distintos estados por los que pasa la animación: rotación, escala y traslación. Estos tres estados se ven reflejados en la gráfica con líneas en forma de *zigzag* para la rotación, una línea recta para la traslación y una semiesfera para la escala (hacia abajo cuando se encoge y hacia arriba cuando se agranda). Sin embargo, la gráfica de la *CPU* es constante, aunque con datos dispersos por la espera de sincronización entre los componentes.

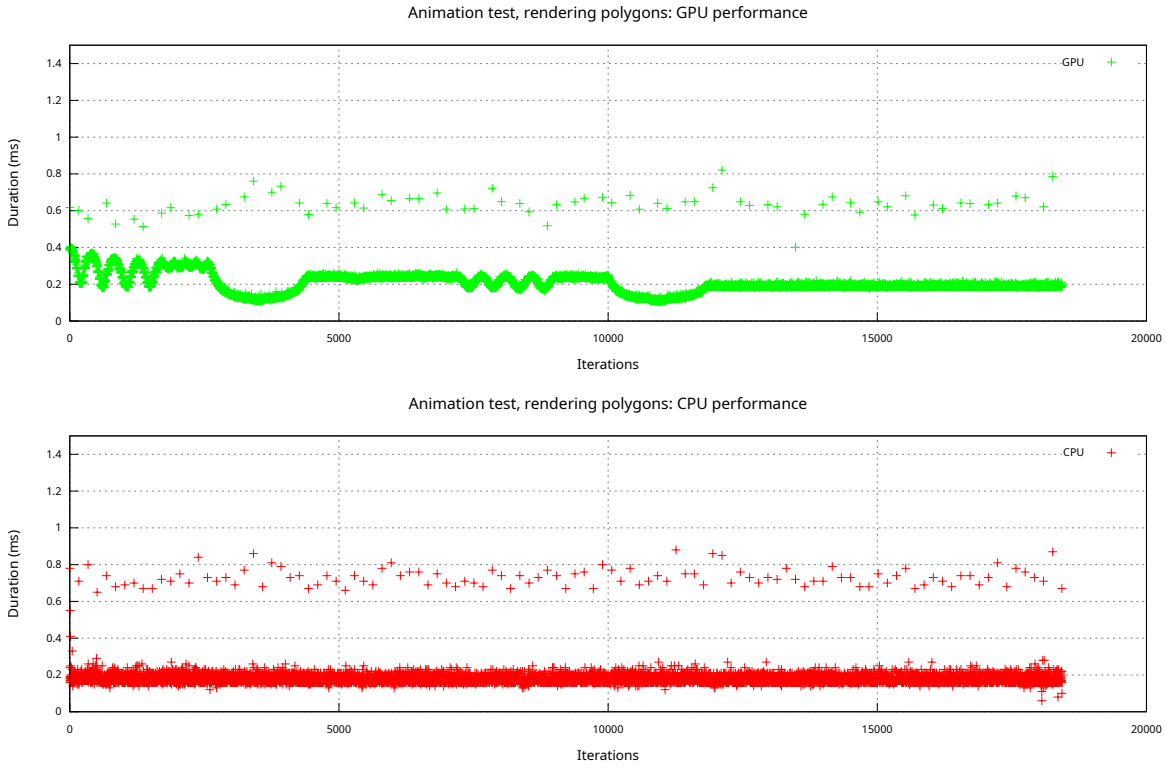


Figura 8.6: Gráficas que representan el rendimiento de la *GPU* y la *CPU* en la prueba de animación durante la renderización.

Media:	0.206815
Mediana	0.192512
Moda	0.163906
IQR	0.047104
Varianza	0.002715
Desviación estándar	0.052101
Covarianza (GPU)	0.000971
Correlación (GPU)	0.419512
Q1	0.190464
Q2	0.192512
Q3	0.237568

Tabla 8.5: Resultados de la renderización en la prueba del polígono: **GPU**

Media	0.177133
Mediana	0.170000
Moda	0.155733
IQR	0.000000
Varianza	0.001975
Desviación estándar	0.044440
Covarianza (GPU)	0.000971
Correlación (GPU)	0.419512
Q1	0.17
Q2	0.17
Q3	0.17

Tabla 8.6: Resultados de la renderización en la prueba del polígono: **CPU**

8.3. Análisis de memoria

Utilizando la herramienta *Valgrind*, especialmente la subherramienta *Massif* [52], se ha generado un informe del uso de memoria, especialmente la petición explícita de memoria (**pila** o *heap)*, dentro de las dos pruebas. Las figuras 8.7 y 8.8 representan el uso de memoria durante la prueba del polígono y animación, respectivamente, siendo el eje horizontal el tiempo y el vertical la memoria usada. Cada uno de los colores en la gráfica representa una función, mencionados en la leyenda, menos el color rojo que cubre al resto en la gráfica que representa el total. Se observa que el uso de memoria alcanza un techo y es constante desde entonces, alcanzando en las dos pruebas los **6MB** de memoria usados. También, las partes del programa que más memoria solicitaron fueron los *drivers* de *NVIDIA* de *OpenGL*, con un uso insignificante frente a las funciones desarrolladas en el proyecto.

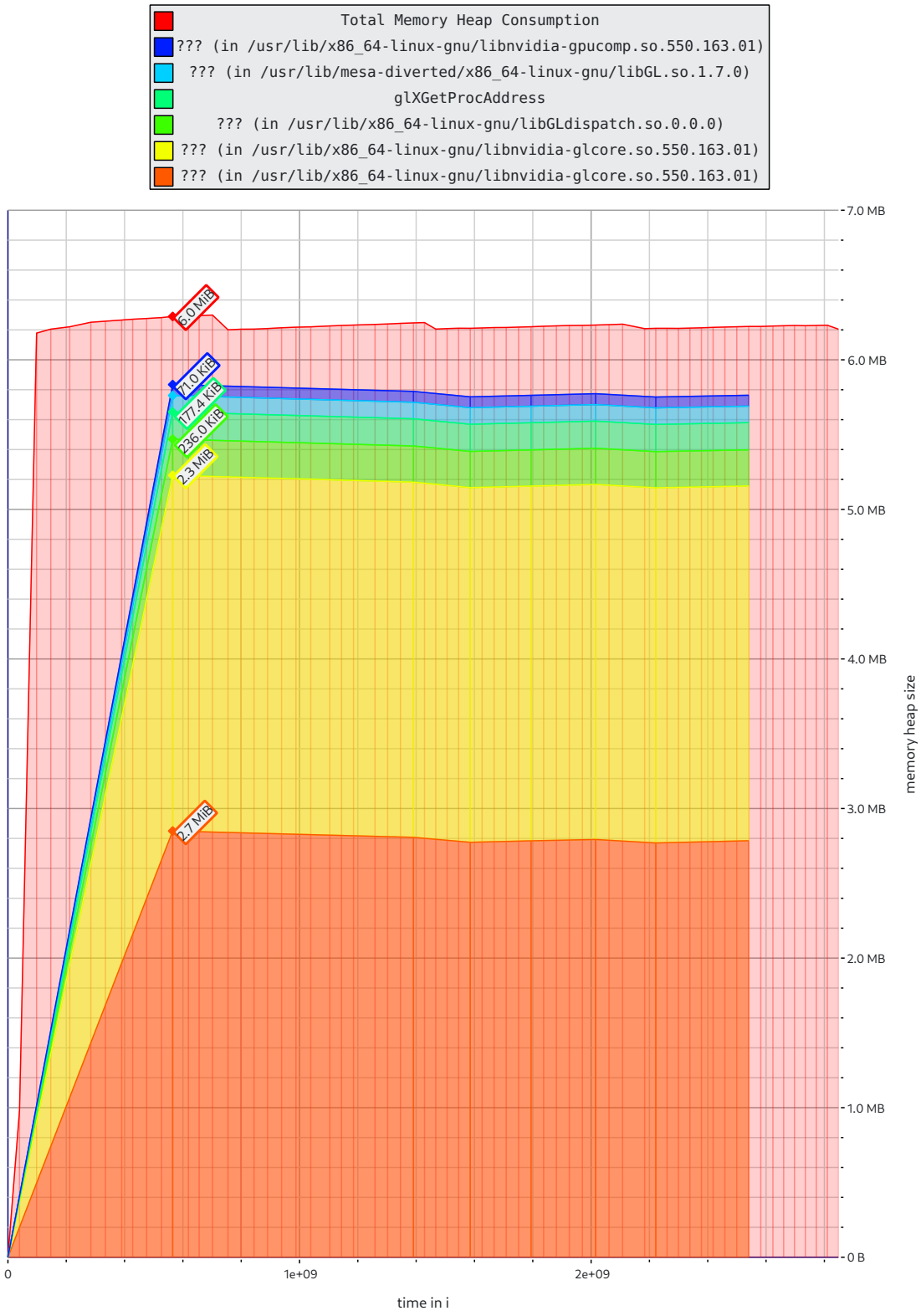


Figura 8.7: Gráfica que representa el uso de memoria en la prueba del polígono

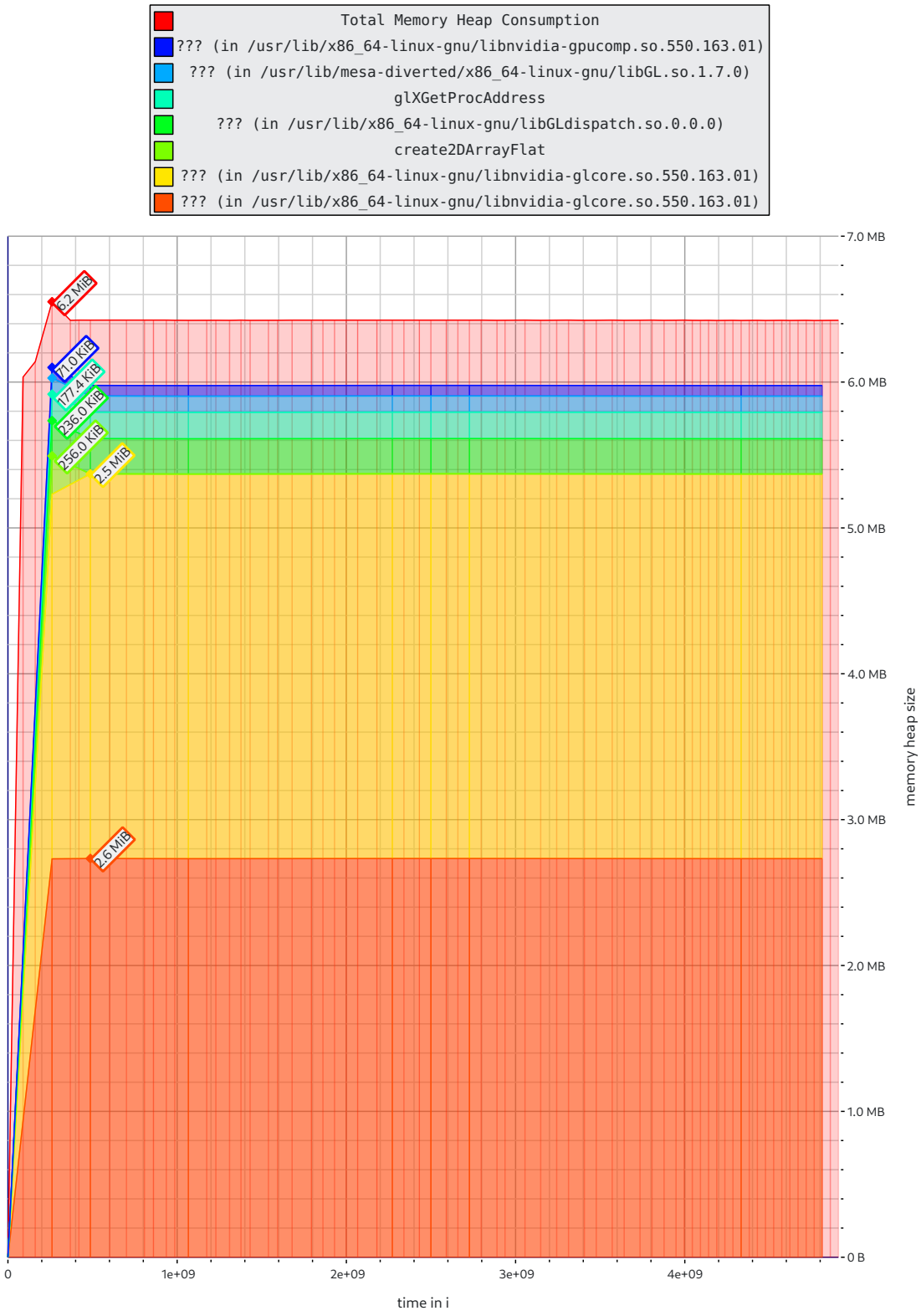


Figura 8.8: Gráfica que representa el uso de memoria en la prueba de animación

8.4. Análisis de CPU

Measure	Animation	Polygon
Task clock (miliseconds)	3,561	3,574
CPUs utilized	0.367	0.915
Page faults	3,609	3,554
Frequency (GHz)	3.187	3.268
Instructions per cycle	1.04	0.77
Branches per second	665.660	550.479
Branch misses percentage	1.21	1.14
Time elapsed (seconds)	9.7086	3.9050
Time user (seconds)	2.2427	1.1099
Time system (seconds)	1.3618	2.4674
Context switches	18,666	1,914
CPU migrations	331	26

Tabla 8.7: Comparación de los resultados de la herramienta *perf* durante la ejecución de las pruebas.

Utilizando la herramienta *perf*, con la subherramienta *stat* [53], se han recopilado datos del rendimiento de la *CPU* durante la ejecución de las pruebas y se han presentado en la tabla 8.7, donde para las pruebas de **animación** y **polígono** se toman diferentes **medidas**.

- El tiempo tomado por ambas tareas es similar, aunque para la animación se haya utilizado un tercio de la cantidad de *CPUs* que en polígono; lógico, ya que en la prueba de animación se divide la carga entre *GPU* y *CPU* más que en la prueba del polígono: el sistema de animaciones y algunas operaciones matriciales son tarea de la *CPU*, mientras que la *GPU* maneja operaciones de memoria y matriciales y renderizado.
- La frecuencia de la *CPU* utilizada en las pruebas es *3.5 GHz*, lo que concuerda con los resultados obtenidos.
- El *IPC* o instrucciones por ciclo son mayores en la prueba de animación.
- Las migraciones de *CPU* se relacionan con un traslado del proceso entre núcleos, semejante a los cambios de contexto, aunque este último se relaciona con el comportamiento del *scheduler* o **planificador** del sistema operativo; ambos valores son más altos en la prueba de animación, debido a mayor número de bibliotecas involucradas y la necesidad de realizar múltiples tareas al mismo tiempo.
- El porcentaje de saltos fallidos es muy bajo, lo que indica una correcta predicción en la ejecución.
- La duración en segundos del usuario junto al sistema sí concuerda en la prueba del polígono, aunque no en la animación, ya que esta última no finaliza en cuanto se completa la renderización de los fotogramas.

8.5. Conclusiones

En este capítulo se han diseñado pruebas sobre los programas creados, midiendo el rendimiento y analizando los resultados. Se ha observado que, según el escenario, puede sufrir más el rendimiento de la *CPU* o la *GPU*: en la prueba de polígonos, era peor el rendimiento de la *CPU*, y en la de animaciones, era peor el rendimiento de la gráfica.

9. Conclusiones

En este proyecto se han desarrollado bibliotecas para el uso de *OpenGL*: bibliotecas de configuración, creación de geometría, animaciones, funciones auxiliares de *C* y gestión de errores. Todas estas bibliotecas se han utilizado en programas: a destacar, el programa de animación, que realiza una serie de transformaciones programables a un polígono regular y se renderiza en pantalla.

Para exprimir y medir las funcionalidades aquí desarrolladas, se han realizado pruebas en las que todas ellas se ven involucradas. Luego, con *R* y *gnuplot*, se toman cómputos numéricos sobre los datos recogidos y gráficas para visualizarlos con detalle.

9.1. Líneas futuras

Este proyecto sólo es una base para los muchos caminos por los que podría expandirse. A continuación, se enunciarán las posibles líneas futuras que se han contemplado: mejoras de rendimiento, nuevas características, integraciones con otros programas y comparativas.

- Mejorar el sistema de animación para permitir geometría tridimensional.
- Capacidad de importación de modelos. Se examinó la posibilidad de incorporar *Assimp* con el fin de probar geometría más compleja y diferente.
- Algoritmos de triangulación, para transformar geometría compleja en triángulos simples, amigables con la *GPU* y más sencillos de procesar.
- Integración con otros programas como extensión y observar ventajas y desventajas. Como caso concreto, *Godot* ofrece una tecnología para interactuar con código *C/C++* llamado *GDExtension* [54], aún en estado experimental; en el caso de ser insuficiente, pueden usarse los módulos personalizados de *C/C++* que también ofrece *Godot* [55].
- Crear comparativas de los tiempos de ejecución respecto a otros programas, como el ya mencionado previamente *Godot*.
- Mejora al sistema de animación, permitiendo fácilmente crear caminos distintos para diferentes geometrías y cargar la animación desde un fichero.
- Mejora a la biblioteca de *shaders*, abstrayendo aún más las funciones principales, como *initAnimationData*, *initGLData* y *initGLShaders*, incluso permitiendo crear *shaders* básicos sin programar *GLSL*.

Bibliografía

[1] Unity. Glosario de unity en el que define el renderer, 2026. URL https://unity.com/glossary/2d-rendering.

[2] Brian W. Kernighan and Dennis M. Ritchie. *The ANSI C Programming Language*. Prentice Hall, 1978.

[3] Khronos. Página de khronos en la que se encuentra la documentación de opengl, 2026. URL https://www.khronos.org/opengl/.

[4] Khronos. Página principal de vulkan, 2026. URL https://www.vulkan.org/.

[5] GNOME. Página principal de gtk, 2026. URL https://www.gtk.org/.

[6] Blender Foundation. Página principal de blender, 2026. URL https://www.blender.org/.

[7] Godot. Página principal de godot, 2026. URL https://www.godotengine.org.

[8] Unity Technologies. Página principal de unity, 2026. URL https://www.unity.com/.

[9] GNU. Página oficial de gnu que explica qué es gnu/linux, 2026. URL https://www.gnu.org/gnu/linux-and-gnu.html.

[10] Linus Torvalds. Página oficial de git, 2026. URL https://git-scm.com/.

[11] Iván Barbero Lay. Repositorio oficial del proyecto, 2026. URL https://gitlab.com/ivabarlay/rendererg12d.

[12] Gitlab Inc. Página oficial de gitlab, 2026. URL https://about.gitlab.com/.

[13] Debian Project. Página oficial de debian, 2026. URL https://www.debian.org/.

[14] Eclipse Foundation. Página oficial de asciidoc, 2026. URL https://asciidoc.org/.

[15] J. Kessenich, Khronos OpenGL ARB Working Group, G. Sellers, and D. Shreiner. *OpenGL® Programming Guide: The Official Guide to Learning OpenGL®, Version 4.5 with SPIR-V, Ninth Edition, 9th edition*. Addison-Wesley Professional, 2016.

[16] CMake. Documentación digital de la herramienta cmake, 2026. URL https://cmake.org/cmake/help/latest/#.

[17] Khronos. Documentación de vulkan sobre el fenómeno llamado aliasing y las técnicas para aliviarlo., 2026. URL https://docs.vulkan.org/samples/latest/samples/performance/msaa/README.html.

[18] Khronos. Página web de khronos, 2026. URL https://www.khronos.org.

[19] NVIDIA. Comparación de opengl y vulkan de nvidia, 2026. URL https://developer.nvidia.com/transitioning-opengl-vulkan.

[20] Godot. Documentación digital del renderizador 2d de godot, 2026. URL https://docs.godotengine.org/en/stable/tutorials/2d/index.html#doc-2d-rendering.

[21] Godot. Documentación digital del renderizador 3d de godot, 2026. URL docs.godotengine.org/en/stable/tutorials/3d/index.html.

[22] Godot. Documentación del nodo direcionallight2d de godot, 2026. URL https://docs.godotengine.org/en/stable/classes/class_directionallight2d.html#class-directionallight2d.

[23] Godot. Documentación del nodo canvasmodulate de godot, 2026. URL https://docs.godotengine.org/en/stable/classes/class_canvasmodulate.html#class-canvasmodulate.

[24] Godot. Documentación del nodo pointlight2d de godot, 2026. URL https://docs.godotengine.org/en/stable/classes/class_pointlight2d.html#class-pointlight2d.

[25] Godot. Documentación del nodo lightoccluder2d de godot, 2026. URL https://docs.godotengine.org/en/stable/classes/class_lightoccluder2d.html#class-lightoccluder2d.

[26] Godot. Documentación del nodo sprite2d de godot, 2026. URL https://docs.godotengine.org/en/stable/classes/class_sprite2d.html#class-sprite2d.

[27] Godot. Documentación del nodo meshinstance2d de godot, 2026. URL https://docs.godotengine.org/en/stable/classes/class_meshinstance2d.html.

[28] Godot. Documentación del nodo animatedsprite2d de godot, 2026. URL https://docs.godotengine.org/en/stable/classes/class_animatedsprite2d.html#class-animatedsprite2d.

[29] Godot. Documentación del nodo animationplayer de godot, 2026. URL https://docs.godotengine.org/en/stable/classes/class_animationplayer.html#class-animationplayer.

[30] Valve. Explicación del concepto de hitbox, 2026. URL https://developer.valvesoftware.com/wiki/Hitbox.

[31] Godot. Ajuste de msaa en godot, 2026. URL https://docs.godotengine.org/en/stable/classes/class_projectsettings.html#class-projectsettings-property-rendering-anti-aliasing-quality-msaa-2d.

[32] Godot. Documentación sobre el lenguaje de shaders de godot, 2026. URL https://docs.godotengine.org/en/stable/tutorials/shaders/shader_reference/shading_language.html.

[33] Godot. Documentación sobre la clase material de godot que permite control sobre shaders, 2026. URL https://docs.godotengine.org/en/stable/classes/class_shadermaterial.html.

[34] Blender Foundation. Documentación digital de cycles, un renderizador de blender, 2026. URL https://developer.blender.org/docs/features/cycles/.

[35] Blender Foundation. Lista de capacidades del renderizador cycles de blender, 2023. URL https://www.cycles-renderer.org/features/.

[36] GNOME. Documentación digital en forma de página web de gtk, 2026. URL https://docs.gtk.org/.

[37] GNOME. Documentación digital de gsk, la dependencia que abstrae la renderización en gtk, 2026. URL https://docs.gtk.org/gsk4/.

[38] GNOME. Documentación sobre la api de renderización que ofrece gsk basada en caminos., 2026. URL https://docs.gtk.org/gsk4/paths.html.

[39] Unity Technologies. Manual digital en forma de página web de unity, 2026. URL https://docs.unity3d.com/Manual/UnityManual.html.

[40] Cppreference. Referencia de funciones de gestión de memoria en c, 2026. URL https://en.cppreference.com/c/memory.

[41] Khronos. Documentación oficial de la especificación de vértices en opengl, 2024. URL https://wikis.khronos.org/opengl/Vertex_Specification.

[42] Khronos. Documentación oficial del flujo de shaders de opengl, 2022. URL https://wikis.khronos.org/opengl/Rendering_Pipeline_Overview.

[43] Khronos. Documentación oficial de glsl, 2026. URL https://wikis.khronos.org/opengl/Core_Language_(GLSL).

[44] Khronos. Documentación oficial de mesa, 2026. URL https://docs.mesa3d.org/.

[45] Nigel Stewart. Página oficial de glew, 2026. URL https://glew.sourceforge.net/.

[46] Freeglut. Página oficial de freeglut, 2026. URL https://freeglut.sourceforge.net/.

[47] Silicon Graphics. Especificación oficial de glut, 1996. URL https://www.opengl.org/resources/libraries/glut/glut-3.spec.pdf.

[48] Eric W Weisstein. Explicación de la convexidad en polígonos de mathworld, 2026. URL https://mathworld.wolfram.com/ConvexPolygon.html.

[49] GNU. Documentación sobre las opciones de optimización del compilador gcc, 2026. URL https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html#index-O3.

[50] Valgrind. Página oficial de la herramienta de análisis valgrind, 2026. URL https://valgrind.org/.

[51] Linux Kernel. Documentación de la herramienta de análisis de linux llamada perf, 2026. URL https://perfwiki.github.io/main/.

[52] Valgrind. Documentación de la herramienta de análisis de memoria massif, 2026. URL https://valgrind.org/docs/manual/ms-manual.html.

[53] Linux Kernel. Documentación de la subherramienta 'stat' de la herramienta perf, 2026. URL https://perfwiki.github.io/main/tutorial/#counting-with-perf-stat.

[54] Godot. Documentación de la tecnología para ejecutar código c/c++ en proyectos de godot llamada gdextension, 2026. URL https://docs.godotengine.org/en/4.4/tutorials/scripting/gdextension/what_is_gdextension.html.

[55] Godot. Documentación de la tecnología para crear módulos personalizados c/c++ en proyectos de godot, 2026. URL https://docs.godotengine.org/en/4.4/contributing/development/core_and_modules/custom_modules_in_cpp.html#doc-custom-modules-in-cpp.